

ROS2 Nav2 PKG 설정



Contents

1 Nav2

1.1 이번에는 정말 할 일이 많습니다.

2 bringup_launch.xml

2.1 arguments

2.2 lifecycle관련 arg 설정

2.3 lifecycle이 뭔가요?

2.4 state의 변환

2.5 Group 태그와namespace

2.6 Component Container

2.7 Component ContainerIsolated

2.8 localization실행

2.9 navigation_launch실행

3 localization_launch.xml

3.1 composable node

3.2 map server 실행

3.3 amcl 실행

3.4 lifecycle 실행

4 AMCL

4.1 잘 설명된 영상

4.2 amcl의 단계

4.3 amcl이 필요로 하는 것

4.4 파티클 필터의 개념

4.5 파티클 필터 설정

4.6 nav2_params.yaml 내 AMCL 섹션 구조 및 주요 설정 범주

4.7 motionparameter

4.8 frame id 및 tf 설정

4.9 센서 설정

4.10 맵 설정

4.11 업데이트조건

5 navigation_launch.xml

5.1 launch 파일 전체 구성

5.1 Controller Server

5.1.1 ControllerServer

5.1.2 일단 controller server를 실행하는 곳 navigation_launch.xml

5.1.3 기본적인 설정 - nav2_params.yaml

5.1.4 progress_checker- nav2_params.yaml

5.1.5 general_goal_checker- nav2_params.yaml

5.1.6 그리고FolloPath설정 하나

5.1.7 Regulated Pure Pursuit란?

5.1.8 회전 우선 매커니즘

5.1.9 회전 우선 매커니즘 설정

5.1.10 경로 추종 관련 설정

5.1.11 lookahead point

5.1.12 Lookahead 파라미터 상세 설정

5.1.13 goal 지점 근처에서 자세 조정

5.1.14 안전 관련

5.1.15 local cost map

5.1.16 local cost map 설정

5.1.17 좌표계 설정

5.1.18 layer 플러그인

5.1.19 inflation layer

5.1.20 static layer

5.2 Velocity Smoother

5.2.1 속도 평활화 효과

5.2.2 velocity smoother 실행

5.2.3 속도 제한 설정

5.2.4 가속도 제한 설정

5.2.5 Odometry 및 타임아웃

5.2.6 Deadband

5.3 Planner server

5.3.1 Planner Server실행

5.3.2 경로 계획 주기

5.3.3 NavFn 파라미터

5.3.4 Global Cost map

5.3.5 Dijkstra와 A*의 차이점 및 use_astar 스위치

5.3.6 글로벌 코스트 맵

5.3.7 Global Costmap 플러그인 레이어 구조

5.3.8 좌표계 설정

5.3.9 플러그인 구조 개요

5.3.10 Static Layer 상세 설정

5.3.11 Obstacle Layer 상세: 센서 통합

5.3.12 Inflation Layer 상세 및 기타 설정

5.4 Smoother Server

5.4.1 구조

5.4.2 Smoother Server 노드 실행 분석

5.5 Waypoint Follower

5.5.1 구조

5.5.2 Waypoint Follower 노드 실행 분석

5.5.3 Waypoint Follower 기본 동작 설정

5.5.4 플러그인 구조: waypoint_task_executor_plugin

5.5.5 WaitAtWaypoint 플러그인 상세

5.6 Behavior Server

5.6.1 구조

5.6.2 실행

5.6.3 플러그인 구조(behavior_plugins)

5.6.4 Behavior Server의 공간 인지 기준이 되는 프레임 및 TF 허용 오차 정의

5.7 BT Navigator

5.7.1 구조

5.7.2 Behavior Tree 기반의 자율주행 미션 오케스트레이션 및 서버 호출 구조

5.7.3 내비게이션 스택의 최상위 오케스트레이터(Orchestrator) 실행 구성

5.7.4 BT Navigator의 좌표계 정의 및 실행 주기 설정

6 ROS2 Navigation 실행해보기

6.1 가상 환경에서 로봇과 센서를 시뮬레이션하기 위해 Gazebo를 구동

6.2 터미널 2: Nav2 Bringup 실행 및 노드 생명주기(Lifecycle) 활성화 확인

6.3 터미널3: 시각화 도구를 실행하여 로봇 상태를 모니터링하고 주행 명령 하달

6.4 RViz2 인터페이스를 통한 목표 지점 지정과 자율주행 실행 과정

7 정말 수고하셨습니다.

ROS2 Nav2 PKG 설정

Exported from Confluence · <https://pinkwink.atlassian.net>

Nav2

이번에는 정말 할 일이 많습니다.

01 bringup launch 파일 분석
환경 설정, Lifecycle Node, 기본 파라미터 구조 이해

02 localization_launch.xml 분석
Map Server, AMCL, Composition 모드 설정

03 AMCL 설정 및 튜닝
확률적 위치 추정, 센서 모델, 입자 필터 튜닝

05 ROS2 navigation 실행해보기
실전 가이드: Gazebo 시뮬레이션부터 RViz 자율주행까지

04 navigation_launch.xml 분석
Nav2 핵심 서버 7단계 심층 분석

- **Part 1** Controller Server
- **Part 2** Velocity Smoother
- **Part 3** Planner Server
- **Part 4** Smoother Server
- **Part 5** Waypoint Follower
- **Part 6** Behavior Server
- **Part 7** BT Navigator

bringup_launch.xml

arguments

주요 인자 (Arguments)

- ✓ **namespace & map**
로봇의 네임스페이스와 사용할 지도 파일 경로(.yaml) 지정
- ✓ **params_file**
Nav2 전체 설정을 담은 YAML 파라미터 파일 경로 설정
- ✓ **use_sim_time**
시뮬레이션 환경(Gazebo 등) 사용 시 `True`로 설정하여 시간 동기화

```
bringup_launch.xml

<?xml version='1.0'?>
<launch>
  <!-- 기본 인자 정의 -->
  <arg name="namespace" default=""/>
  <arg name="map" default=""/>
  <arg name="use_sim_time" default="False"/>

  <!-- 파라미터 파일 경로 -->
  <arg name="params_file"
        default="$(find-pkg-share pinky_navigation)/params/nav2_params.yaml"/>

  <arg name="autostart" default="True"/>
  <arg name="container_name" default="nav2_container"/>
  <arg name="use_composition" default="True"/>
  <arg name="use_respawn" default="False"/>
  <arg name="log_level" default="info"/>
  ...
</launch>
```

lifecycle 관련 arg 설정

🔄 Lifecycle 관리 대상

ROS2 관리형 노드(Managed Node) 목록. **Lifecycle Manager**에 의한 순차적 초기화 및 활성화 수행.

노드 그룹 분류

📄 Local (Localization)

`map_server` (지도), `amcl` (위치 추정) 포함

📄 Nav (Navigation)

경로 계획(`planner`), 제어(`controller`), 행동 트리(`bt_navigator`) 등 핵심 주행 노드 구성

```
bringup_launch.xml

<!-- ... 이전 코드 생략 ... -->

<!-- Localization 관련 노드 목록 -->
<arg name="lifecycle_nodes_local"
      default="['map_server', 'amcl']"/>

<!-- Navigation 관련 노드 목록 -->
<arg name="lifecycle_nodes_nav"
      default="['controller_server',
                'planner_server',
                'behavior_server',
                'bt_navigator',
                'waypoint_follower',
                'velocity_smoother']"/>

<!-- ... 이후 코드 생략 ... -->
```

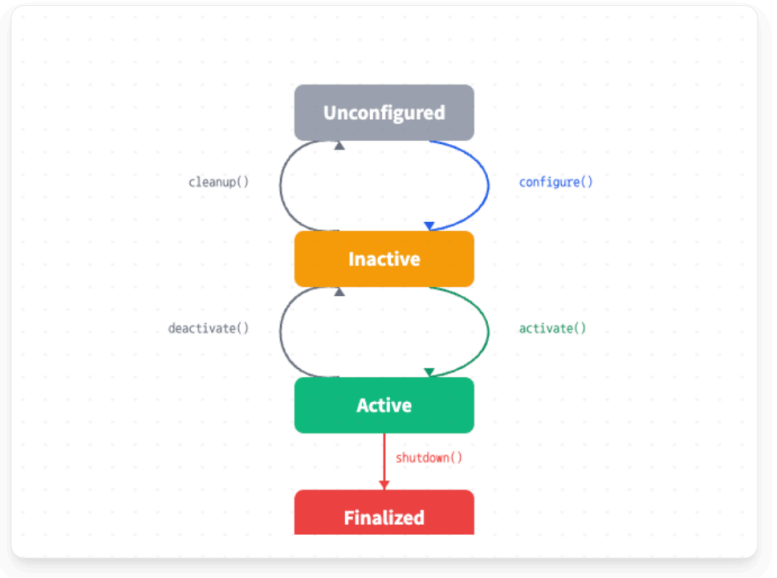
lifecycle이 뭔가요?

주요 상태 (Primary States)

- 1 Unconfigured** (미구성)
노드 생성 직후. 파라미터 미로드 상태.
- 2 Inactive** (비활성)
구성(Configure) 완료. 파라미터 보유, 데이터 처리/발행 미수행.
- 3 Active** (활성)
완전 가동 상태. 데이터 수신·처리·발행 등 주요 기능 수행.
- 4 Finalized** (종료)
노드 종료 직전 상태. 리소스 해제 완료.

Nav2의 특징

Nav2 핵심 서버(Planner, Controller 등) Lifecycle Node 구현 및 체계적 관리.



state의 변환

시스템의 지휘자 (Conductor)

다중 노드 일괄 관리

`map\_server`, `planner`, `controller` 등 개별 동작 노드 상태 중앙 관리

순차적 기동 (Sequential Startup)

의존성 정의 순서에 따른 노드 `Configure` 및 `Activate` 수행 (예: 맵 서버 준비 후 플래너 동작)

Autostart 기능

`autostart=True` 설정 시 시스템 시작과 동시 활성 상태(Active) 자동 전이 및 즉시 사용 가능 상태 전환

```
> Lifecycle Manager Log Simulation

# 1. 관리 대상 노드 목록 로드
[INFO] [lifecycle_manager]: Creating lifecycle nodes: ['map_server', 'amcl', 'controller_server', ...]

# 2. 상태 설정 (Unconfigured → Inactive)
[INFO] [lifecycle_manager]: Configuring map_server
[INFO] [lifecycle_manager]: Configuring amcl
[INFO] [lifecycle_manager]: Configuring controller_server
... (순차적으로 모든 노드 설정 완료)

# 3. 노드 활성화 (Inactive → Active)
[INFO] [lifecycle_manager]: Activating map_server ✓
[INFO] [lifecycle_manager]: Activating amcl ✓
[INFO] [lifecycle_manager]: Activating controller_server ✓

>> The Nav2 system is active and ready!
```

Group 태그와 namespace

- group은 “실행”의 단위가 아니라 “설정 범위”의 단위

핵심 기능

노드 그룹화 (Grouping)

<group> 태그 활용: 연관 노드 및 include 파일 블록화. 조건부 실행 (if/unless) 일괄 적용 유용

네임스페이스 격리 (Isolation)

<push-ros-namespace> 활용: 그룹 내 모든 리소스(토픽·서비스·액션) 이름 앞 접두어 부여

예: cmd_vel → /robot1/cmd_vel

멀티 로봇 지원

네임스페이스 격리: 동일 코드 다중 로봇 동시 실행 및 통신 간섭 방지

```
<group>
  <!-- 1. 네임스페이스 밀어넣기 (Push) -->
  <!-- namespace 변수가 'robot1'이라면 -->
  <!-- 이 그룹 내의 모든 토픽은 /robot1/... 로 시작됨 -->
  <push-ros-namespace namespace="$(var namespace)"/>

  <!-- 2. 컨테이너 노드 실행 -->
  <!-- 실제 노드 이름: /robot1/nav2_container -->
  <node pkg="rclcpp_components" ... </node>

  <!-- 3. 다른 Launch 파일 포함 -->
  <!-- 포함된 파일 내부의 노드들도 네임스페이스 상속 -->
  <include file="$(find-pkg-share ...)/localization_launch.xml">
    ...
  </include>
</group>
```

Component Container

핵심 개념

Component Container

ROS2 노드(Component) 실행을 위한 **런타임 호스트 프로세스**. 개별 노드를 실행 파일이 아닌 라이브러리 형태로 생성 후 컨테이너 적재

오버헤드 감소

수십 개의 노드를 **단일 프로세스** 내에서 통합 실행. CPU/메모리 자원 소모 절감 및 컨텍스트 스위칭 비용 최소화

Isolated 모드

component_container_isolated 사용: 각 컴포넌트별 전용 Executor(스레드 풀) 할당. 특정 노드의 부하 전파 방지 및 독립성 보장

```
<!-- 컨테이너 프로세스 실행 -->
<node pkg="rclcpp_components"
  exec="component_container_isolated"
  name="$(var container_name)"
  output="screen"
  args="--ros-args --log-level $(var log_level)">

  <!-- 컨테이너 자체에 필요한 파라미터 로딩 -->
  <param from="$(var params_file)"/>
  <param name='map' value="$(var map)"/>
  <param name='autostart' value="$(var autostart)"/>
  <param name='use_sim_time' value="$(var use_sim_time)"/>

  <!-- TF Topic Remapping - 컨테이너 내 모든 노드에 적용됨 -->
  <remap from="/tf" to="tf"/>
  <remap from="/tf_static" to="tf_static"/>
</node>
```

Component Container Isolated

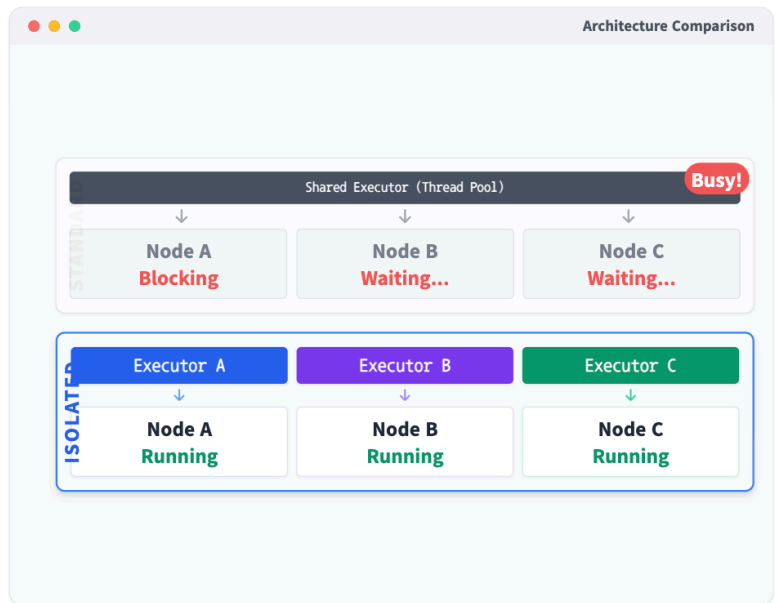
핵심 개념

기본 컨테이너: 모든 노드가 단일 Executor 공유.

반면, **Isolated 컨테이너**: 각 컴포넌트마다 **전용 Executor** 할당.

주요 이점 (Benefits)

- 부하 격리 (Load Isolation)
특정 노드(예: 복잡한 연산) 과부하 시에도 타 노드(예: 센서 처리)는 전용 스레드에서 안정적 실행 보장.
- 병목 현상 방지
공유 스레드 풀 포화로 인한 중요 작업의 대기(Blocking) 상태 방지.



localization 실행

Localization 모듈 실행

`localization_launch.xml` 파일 include, Map Server 및 AMCL 노드 실행

핵심 전달 인자

- 리소스 공유 (Container)
`container_name` 전달, 기 생성 컴포넌트 컨테이너 내 노드 실행
- 지도 및 설정 (Map & Params)
상위 `map` 경로 및 `params_file` 하위 런치 파일로 전달
- 라이프사이클 (Lifecycle Nodes)
로컬라이제이션 그룹 해당 노드 목록(`lifecycle_nodes_local`) 명시적 전달

```
bringup_launch.xml

<include file="$(find-pkg-share pinky_navigation)/launch/localization_launch.xml">
  <!-- 기본 설정 전달 -->
  <arg name="params_file" value="$(var params_file)"/>
  <arg name="map" value="$(var map)"/>
  <arg name="use_sim_time" value="$(var use_sim_time)"/>
  <arg name="autostart" value="$(var autostart)"/>

  <!-- 컴포지션 및 리소스 설정 -->
  <arg name="use_composition" value="$(var use_composition)"/>
  <arg name="use_respawn" value="$(var use_respawn)"/>
  <arg name="container_name" value="$(var container_name)"/>

  <!-- 관리 대상 노드 목록 전달 -->
  <arg name='lifecycle_nodes' value="$(var lifecycle_nodes_local)"/>
</include>
```

navigation_launch 실행

📍 내비게이션 스택 실행

`navigation_launch.xml` 호출: 경로 계획(Planner) · 제어 (Controller) · 행동 트리(BT) 등 핵심 노드 실행

주요 전달 인자 (Args)

- 🔗 **container_name**
컴포넌트 컨테이너 이름 공유: 단일 프로세스 내 효율적 실행
- 🔗 **lifecycle_nodes**
내비게이션 노드 리스트(lifecycle_nodes_nav) 전달 및 상태 관리 위임
- 🔗 **params_file & autostart**
설정 파일 경로 및 자동 시작 여부 전달

... (Localization Include 이후)

```
<include file="$(find-pkg-share pinky_navigation)/launch/navigation_launch.xml">
  <arg name="params_file" value="$(var params_file)"/>
  <arg name="use_sim_time" value="$(var use_sim_time)"/>
  <arg name="autostart" value="$(var autostart)"/>
  <arg name="use_composition" value="$(var use_composition)"/>
  <arg name="use_respawn" value="$(var use_respawn)"/>
  <arg name="container_name" value="$(var container_name)"/>

  <!-- 내비게이션 관련 노드 리스트 전달 -->
  <arg name='lifecycle_nodes'
        value="$(var lifecycle_nodes_nav)"/>
</include>
</group>

</launch>
```

localization_launch.xml

composable node

실행 모드 비교



Composition 모드

단일 프로세스 내 여러 노드 통합 실행

`load_composable_node if="$(var use_composition)"`



일반 노드 모드

각 노드를 개별 프로세스로 분리 실행

`group unless="$(var use_composition)"`

📌 핵심 차이점

- 메모리 사용량 및 통신 오버헤드 최적화 여부
- 디버깅 편의성 vs 런타임 성능 트레이드오프

```
<!-- 1. Composition 모드: use_composition이 True일 때 실행 -->
<load_composable_node target="$(var container_name)"
                      if="$(var use_composition)">
  <!-- nav2_map_server, nav2_amcl 등을 컨테이너에 적재 -->
  <composable_node ... />
  <composable_node ... />
</load_composable_node>

<!-- 2. 일반 노드 모드: use_composition이 False일 때 실행 -->
<group unless="$(var use_composition)">
  <push-ros-namespace namespace="$(var namespace)"/>

  <!-- nav2_map_server, nav2_amcl 등을 개별 프로세스로 실행 -->
  <node pkg="nav2_map_server" exec="map_server" ... >
    ... </node>
</group>
```

map server 실행

주요 구성 요소

역할 (Role)

YAML 파일 로딩 및 OccupancyGrid 토픽 발행

패키지 정보

nav2_map_server 패키지 사용

Plugin: nav2_map_server::MapServer

핵심 파라미터

yaml_filename: 지도 파일 경로 설정

TF 리매핑 (Isolation)

/tf → tf 변환으로 컨테이너 내부 통신 격리

```
localization_launch.xml

<composable_node pkg="nav2_map_server"
  plugin="nav2_map_server::MapServer"
  name="map_server">

  <!-- 파라미터 로딩 -->
  <param from="$(var params_file)"/>
  <param name='use_sim_time' value="$(var use_sim_time)"/>

  <!-- 지도 경로 설정 -->
  <param name='yaml_filename' value="$(var map)"/>

  <!-- TF 리매핑 (컨테이너 내부 통신용) -->
  <remap from="/tf" to="tf"/>
  <remap from="/tf_static" to="tf_static"/>

</composable_node>
```

amcl 실행

핵심 기능 (Core Function)

확률적 위치 추정

LIDAR 센서 데이터와 지도를 매칭하여 로봇의 현재 위치(Pose) 계산

플러그인 로딩

nav2_amcl::AmclNode 를 컨테이너에 직접 로드하여 통신 지연 최소화

주요 설정 (Configuration)

파라미터 적용

파티클 수, 초기 위치 분산 등 튜닝 값을 params_file 에서 로드

TF 리매핑 (Remapping)

다중 로봇 환경 고려, /tf 토픽을 로컬 네임스페이스로 변경

```
localization_launch.xml

<!-- AMCL 컴포저블 노드 -->
<composable_node pkg="nav2_amcl"
  plugin="nav2_amcl::AmclNode"
  name="amcl">

  <!-- 파라미터 파일 로드 -->
  <param from="$(var params_file)"/>

  <!-- 시뮬레이션 시간 동기화 -->
  <param name='use_sim_time' value="$(var use_sim_time)"/>

  <!-- TF 리매핑 -->
  <remap from="/tf" to="tf"/>
  <remap from="/tf_static" to="tf_static"/>

</composable_node>
```

lifecycle 실행

주요 기능 및 설정

☰ 상태 전이 관리 (State Transition)

등록된 노드들의 Unconfigured → Inactive → Active 상태 전환 순차 제어

🔧 자동 시작 (Autostart)

`autostart=True` 설정 시 시스템 기동과 동시에 활성화 상태로 자동 진입

☰ 대상 노드 목록 (Node Names)

제어할 노드 리스트 전달 (`['map_server', 'amcl']`)

📌 Nav2 구조의 핵심

개별 노드가 아닌 매니저를 통한 중앙 집중식 상태 관리가 Nav2 아키텍처의 특징

```
localization_launch.xml

<!-- Lifecycle Manager 컴포저블 노드 -->
<composable_node pkg="nav2_lifecycle_manager"
  plugin="nav2_lifecycle_manager::LifecycleManager"
  name="lifecycle_manager_localization">

  <!-- 시뮬레이션 시간 동기화 -->
  <param name="use_sim_time" value="$(var use_sim_time)"/>

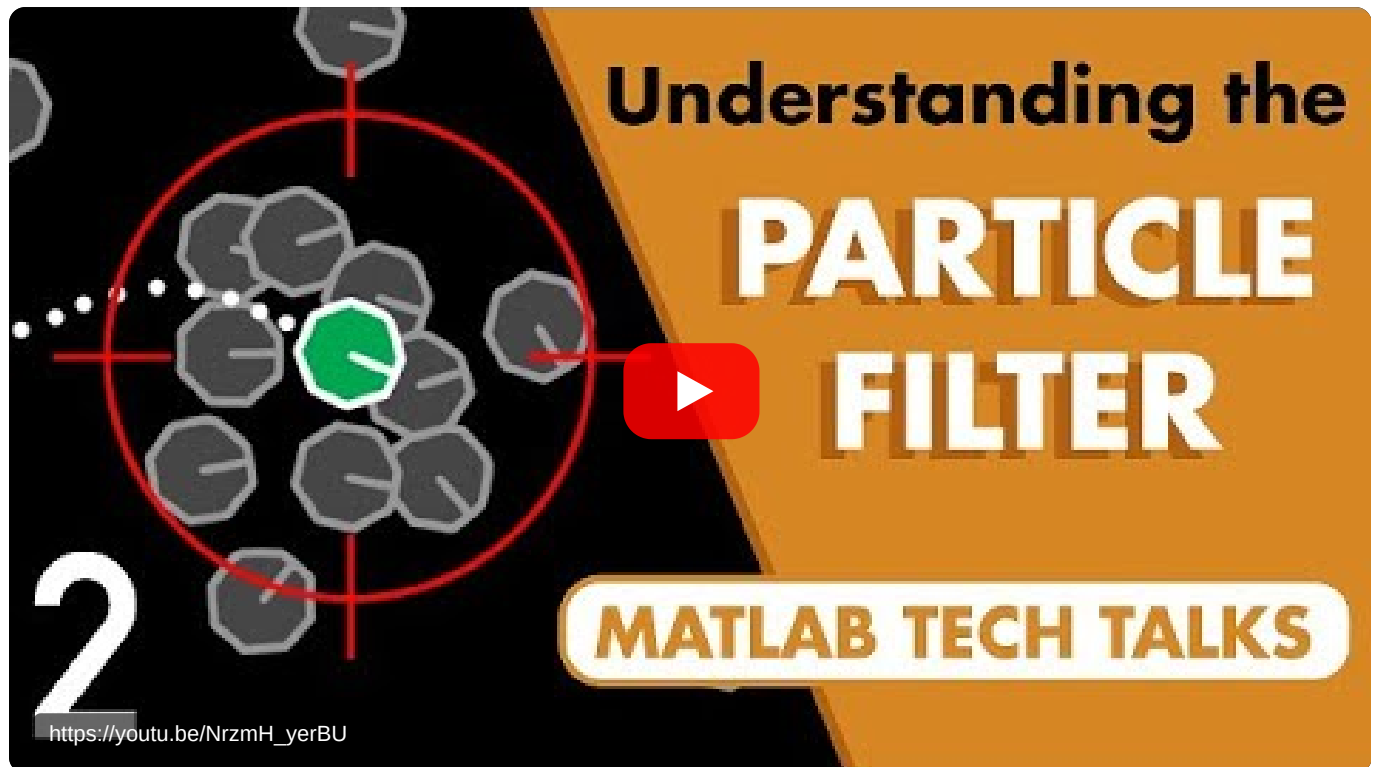
  <!-- 자동 시작 옵션 (True/False) -->
  <param name="autostart" value="$(var autostart)"/>

  <!-- 관리 대상 노드 목록 -->
  <param name="node_names" value="$(var lifecycle_nodes)"/>

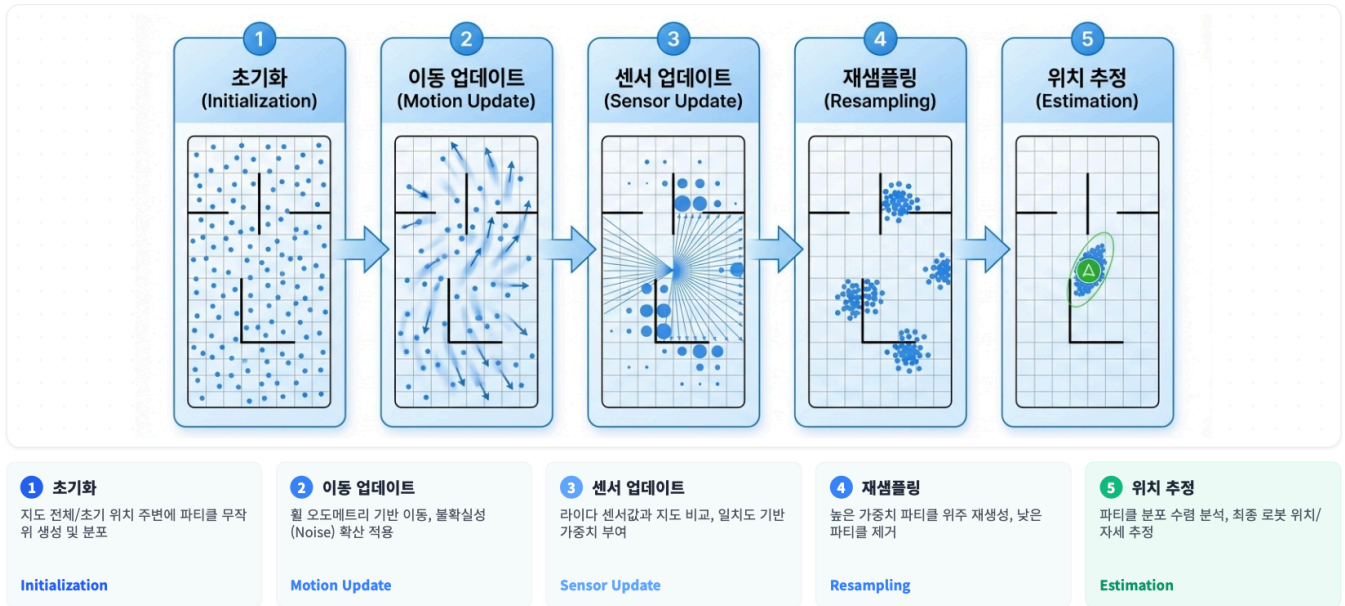
</composable_node>
```

AMCL

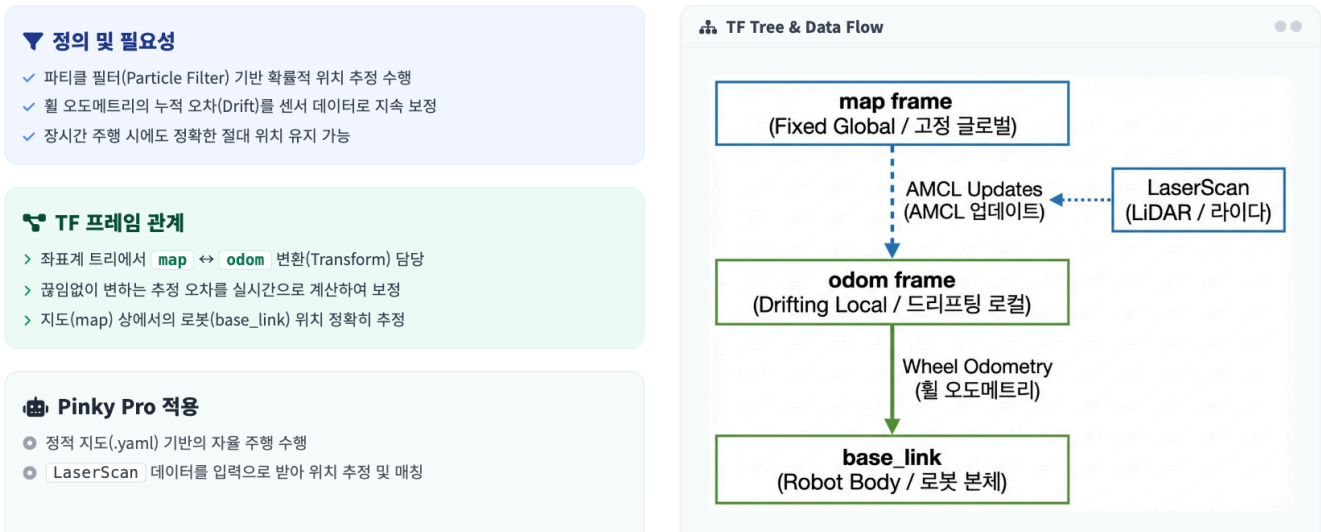
잘 설명된 영상



amcl의 단계



amcl이 필요로 하는 것



파티클 필터의 개념

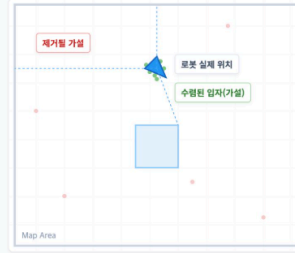
핵심 요약

- ✓ **확률 분포 표현:** 수백~수천 개 파티클 사용
- ✓ **위치 가설:** 각 파티클 = 로봇 위치 (x, y, yaw)
- ✓ **신뢰도 갱신:** LiDAR 데이터 비교로 가중치 부여

Pinky Pro 적용 사례

- LaserScan 입력**
2D LiDAR 데이터를 관측 모델로 사용하여 지도와 매칭
- 실시간 보정**
주행 중 발생하는 휠 오도메트리의 누적 오차(Drift) 해소
- 정확도 유지**
복잡한 환경에서도 정적 지도 기반 정밀 위치 추정 수행

MCL (Monte Carlo Localization) Visualization



핵심 메커니즘 (CORE MECHANISM)

- 초기화 (Initialization)**
지도상에 무작위 입자 생성 및 배포 (Global Localization)
- 예측 (Prediction)**
오도메트리 기반 입자 이동 (Motion Model 적용)
- 업데이트 (Update)**
센서값과 비교하여 가중치 계산 (Sensor Model 적용)

파티클 필터 설정

입자 수 제어 (Performance)

- min_particles / max_particles**
최소/최대 입자 수 지정 (정확도와 CPU 부하 간 트레이드오프 조절)

KLD 샘플링 (Accuracy)

- pf_err & pf_z**
허용 오차(err)와 신뢰 수준(z). 작을수록 더 많은 입자 생성

리샘플링 및 복구 (Recovery)

- resample_interval**
필터 업데이트 n회마다 리샘플링 수행 (빈도 조절)
- recovery_alpha_fast / slow**
위치 소실 시 랜덤 입자 주입 비율 (Augmented MCL)

nav2_params.yaml로 ros_parameters:

```
# 입자 수 범위 (시스템 성능에 큰 영향)
min_particles: 500
max_particles: 2000
# KLD(Kullback-Leibler Divergence) 샘플링
pf_err: 0.05 # 허용 오차 (작을수록 입자 증가)
pf_z: 0.99 # 신뢰 수준 (표준 정규분포 상위 분위수)
# 리샘플링 실행 주기 (업데이트 횟수 기준)
resample_interval: 1

# 위치 복구 파라미터 (0.0 = 비활성)
# 납치된 로봇 문제(Kidnapped Robot) 해결용
recovery_alpha_fast: 0.0 # 단기 평균
recovery_alpha_slow: 0.0 # 장기 평균

# 추정된 자세 저장 주기 (Hz)
save_pose_rate: 0.5
```

nav2_params.yaml 내 AMCL 섹션 구조 및 주요 설정 범주

설정 구조 및 위치

YAML 계층 구조
params 파일 내 `amcl` > `ros__parameters` 하위에 정의

핵심 목적
정적 지도 기반 Localization을 위한 확률적 파라미터 제어

주요 파라미터 범주

Motion Model

오도메트리 노이즈 특성

Sensor Model

LiDAR 데이터 처리 방식

Particle Filter

샘플 수 및 리샘플링

Frame & TF

좌표계 및 변환 설정

주의 사항

코드 블록의 값은 예시이며, 실제 로봇 센서 및 환경에 맞춰 튜닝 필수

```
amcl:
  ros__parameters:
    # 1. Motion Model (이동 모델)
    alpha1: ...
    # ...

    # 2. Frame ID & TF (좌표계)
    ...

    # 3. Sensor Model (센서 모델)
    ...

    # 4. Particle Filter (입자 필터)
    min_particles: 500
    max_particles: 2000
    update_min_d: 0.15
    initial_pose: [0.0, 0.0, 0.0]
```

motion parameter

파라미터 역할 (Role)

- ✓ **오도메트리 노이즈 모델링**
바퀴 미끄러짐, 바닥 마찰 등으로 인한 이동 오차를 확률적으로 반영
- ✓ **파티클 확산 제어**
노이즈 계수가 클수록 파티클이 넓게 퍼짐 (위치 불확실성 증가)

노이즈 계수 상세 (Alpha)

alpha1 회전 운동 시 발생하는 **회전 오차**
alpha2 평행 이동 시 발생하는 **회전 오차**
alpha3 평행 이동 시 발생하는 **이동 오차**
alpha4 회전 운동 시 발생하는 **이동 오차**
alpha5 전방향(Omni) 로봇용 추가 오차 계수

```
ros__parameters:
  # --- Motion Model Configuration ---
  # [회전 노이즈] 회전 동작이 회전 오차에 미치는 영향
  alpha1: 0.2
  # [회전 노이즈] 평행 이동이 회전 오차에 미치는 영향
  alpha2: 0.2
  # [이동 노이즈] 평행 이동이 이동 오차에 미치는 영향
  alpha3: 0.2
  # [이동 노이즈] 회전 동작이 이동 오차에 미치는 영향
  alpha4: 0.2
  # [Omni 전용] 전방향 로봇을 위한 추가 노이즈 계수
  alpha5: 0.2

  # 로봇 구동 방식 설정
  # "nav2_amcl::DifferentialMotionModel" (차동 구동)
  # "nav2_amcl::OmniMotionModel" (전방향 구동)
  robot_model_type: "nav2_amcl::DifferentialMotionModel"
```

frame id 및 tf 설정

좌표계 프레임 (Frame IDs)

-  **base_frame_id**
로봇 본체 기준 프레임 (예: base_footprint)
-  **global_frame_id**
전역 고정 좌표계 (예: map)
-  **odom_frame_id**
오도메트리 기준 좌표계 (예: odom)

TF 동작 설정

-  **tf_broadcast**
map ↔ odom 변환 TF 발행 활성화/비활성화
-  **transform_tolerance**
TF 변환 시간 동기화 허용 오차 (초 단위)

```
amcl:
  ros__parameters:
    base_frame_id: "base_footprint"

    global_frame_id: "map"

    odom_frame_id: "odom"

    # AMCL이 map->odom 변환을 게시할지 여부
    tf_broadcast: true

    # 변환 시간 허용 오차 (System Latency 대응)
    transform_tolerance: 1.0
```

센서 설정

주요 설정 포인트

-  **모델 유형 및 거리**
"likelihood_field" 방식 권장 (거리장 기반 스코어링)
최대/최소 유효 범위 설정으로 센서 노이즈 필터링
-  **최대 빔 수 (Max Beams)**
업데이트 단계에서 샘플링할 빔 개수 제한
계산 부하와 정확도 간의 트레이드오프 조정
-  **빔 스킵 (Beam Skip)**
동적 장애물 등 이상치(Outlier) 무시 옵션
지도와 불일치하는 빔 비율이 임계값 초과 시 스킵

```
amcl:
  ros__parameters:
    # --- 레이저 모델 및 범위 설정 ---
    laser_model_type: "likelihood_field"
    laser_min_range: -1.0 # -1.0: 센서의 최소 범위 사용
    laser_max_range: 100.0 # 최대 유효 거리
    laser_likelihood_max_dist: 2.0
    max_beams: 60 # 빔 샘플링 수 (높을수록 부하 증가)

    # --- 빔 스킵 (Beam Skip) 설정 ---
    do_beamskip: false # true 설정 시 기능 활성화
    beam_skip_threshold: 0.3 # 스킵할 빔의 비율 임계값
    beam_skip_distance: 0.5 # 장애물과의 거리 차이 허용치
    beam_skip_error_threshold: 0.9

    scan_topic: scan # 구독할 LaserScan 토픽명
```

맵 설정

🔗 일치 모델 (Hit)

- **z_hit** (가중치)
지도와 일치하는 측정값 비중 (가장 높게 설정)
- **sigma_hit** (표준편차)
위치 불확실성 허용 범위 (작을수록 정밀도 요구)

🔗 노이즈 및 예외 처리

- **z_rand / z_max**
랜덤 잡음 및 최대 거리(수신 실패) 가중치
- **z_short / lambda_short**
동적 장애물(지도에 없음)에 의한 짧은 거리 측정 반영

nav2_params.yaml

```
# Likelihood Field 모델 파라미터
z_hit: 0.5          # 일치(Hit) 항 가중치 (Main)
z_rand: 0.5         # 랜덤 노이즈 항 가중치
sigma_hit: 0.2      # 가우시안 표준편차 (단위: m)

# 추가 파라미터 (Beam Model 등에서 주로 사용)
z_short: 0.05       # 짧은 거리(Short) 항 가중치
lambda_short: 0.1    # 지수 분포 감쇠 계수
z_max: 0.05         # 최대 거리(Max Range) 항 가중치

# -----
# Tip: 모든 z_* 가중치의 합은 1.0이 되는 것이 이상적
# likelihood_field 사용 시 z_hit + z_rand 조합이 일반적
```

💡 초기 튜닝 시 **z_hit**과 **z_rand**의 비율 조절이 성능에 가장 큰 영향을 줌

업데이트 조건

🔄 업데이트 조건 (Update Thresholds)

- **update_min_d** (이동 거리)
로봇이 최소 이 거리만큼 이동했을 때 필터 업데이트 수행 (단위: m). 불필요한 연산 방지
- **update_min_a** (회전 각도)
로봇이 최소 이 각도만큼 회전했을 때 필터 업데이트 수행 (단위: rad)

📍 초기 위치 설정 (Initial Pose)

- **initial_pose & set_initial_pose**
초기 위치 [x, y, yaw] 지정 및 파라미터 적용 여부 설정. Rviz '2D Pose Estimate' 대신 자동 초기화 가능

nav2_params.yaml

```
amcl:
  ros_parameters:
    # --- 필터 업데이트 임계값 ---
    # 너무 작으면 연산 부하 증가, 크면 추정 지연
    update_min_d: 0.15 # 미터 (Linear)
    update_min_a: 0.1  # 라디안 (Angular, 약 5.7도)

    set_initial_pose: true
    initial_pose: [0.0, 0.0, 0.0]

    tf_broadcast: true
    transform_tolerance: 1.0
```

💡 **운영 팁**
초기 위치가 실제 로봇 위치와 가까울수록 입자 수렴 속도가 빨라지며, 잘못된 위치 추정(Local Minima)에 빠질 확률이 줄어 듭니다

navigation_launch.xml

launch 파일 전체 구성

최상위 제어 (Top Level)

BT Navigator

행동 트리(Behavior Tree) 기반 내비게이션 총괄 제어

핵심 서버군 (Core Servers)

Planner / Controller / Smoother

전역 경로 계획, 지역 경로 추종, 궤적 평활화 수행

Behavior / Waypoint Follower

회전·백업 등 회복 행동 및 다중 경유지 관리

출력 및 관리 (Output & Mgmt)

Velocity Smoother

속도 명령(cmd_vel) 가감속 제한 및 평활화

Lifecycle Manager

모든 노드의 상태(Active/Inactive) 일괄 제어

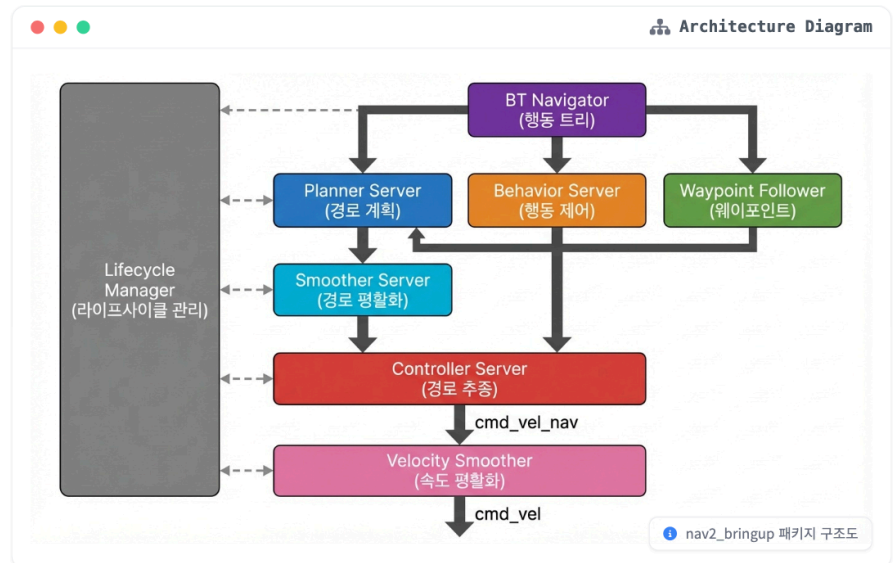
Costmap 실행 위치

Global planner_server 노드에서 실행

전역 경로 계획용 정적 맵 기반

Local controller_server 노드에서 실행

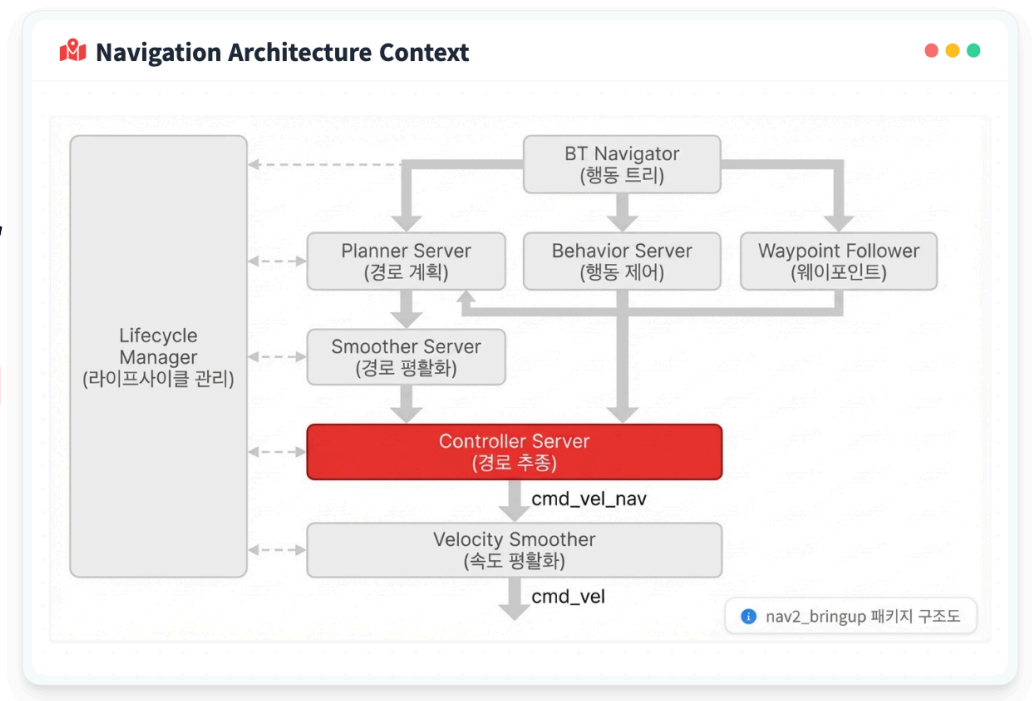
장애물 회피용 실시간 센서 기반



Controller Server

Controller Server

- controller server는 모터 제어가 아니라, 로봇 제어기



CHAPTER 4 - PART 1

Controller Server

경로 추종

계획된 경로를 따라
로봇을 정밀하게 제어하는
핵심 컴포넌트

일단 controller server를 실행하는 곳 navigation_launch.xml


```

13     <arg name="lifecycle_nodes" default="['controller_server',
21     <load_composable_node target="$(var container_name)" if="$(var use_composition)">
22         <composable_node pkg="nav2_controller" plugin="nav2_controller::ControllerServer"
23             <param from="$(var params_file)"/>
24             <param name="use_sim_time" value="$(var use_sim_time)"/>
25             <param name="autostart" value="$(var autostart)"/>
26
27             <remap from="/tf" to="tf"/>
28             <remap from="/tf_static" to="tf_static"/>
29             <remap from="cmd_vel" to="cmd_vel_nav"/>
30     </composable_node>

```

기본적인 설정 - nav2_params.yaml

```

70 controller_server:
71   ros__parameters:
72     controller_frequency: 20.0
73     costmap_update_timeout: 0.30
74     min_x_velocity_threshold: 0.001
75     min_y_velocity_threshold: 0.5
76     min_theta_velocity_threshold: 0.001
77     failure_tolerance: 0.3

```

- **controller_frequency** : cmd_vel을 생성하는 주기
- **failure_tolerance** : 컨트롤러가 경로를 못 따라갈때 허용하는 대기 시간

progress_checker - nav2_params.yaml

```

70 controller_server:
71   ros__parameters:
87     progress_checker:
88       plugin: "nav2_controller::SimpleProgressChecker"
89       required_movement_radius: 0.5
90       movement_time_allowance: 10.0

```

- 일정 시간 동안, 일정 거리 이상 움직이지 않으면, 즉, costmap은 살아있는데 로봇이 제자리 → 실패
- 현재 설정) 10초동안 0.5m 반경을 이동하지 못하면 실패

general_goal_checker - nav2_params.yaml

```

70 controller_server:
71   ros__parameters:
93     general_goal_checker:
94       stateful: True
95       plugin: "nav2_controller::SimpleGoalChecker"
96       xy_goal_tolerance: 0.25
97       yaw_goal_tolerance: 0.25

```

- 목표지점으로부터 반경 0.25m, 자세 0.25 라디안 안에 들어오면 성공

그리고 FolloPath 설정 하나

```

70 controller_server:
71   ros__parameters:

100   FollowPath:
101     plugin: "nav2_regulated_pure_pursuit_controller::RegulatedPurePursuitController"
102
103     # 기본 속도들 (Pinky 상황 맞게 나중에 조정 가능)
104     desired_linear_vel: 0.20      # [m/s]
105     max_linear_accel: 0.5         # [m/s^2]
106     max_linear_decel: 0.5         # [m/s^2]
107     max_angular_vel: 1.0          # [rad/s]

```

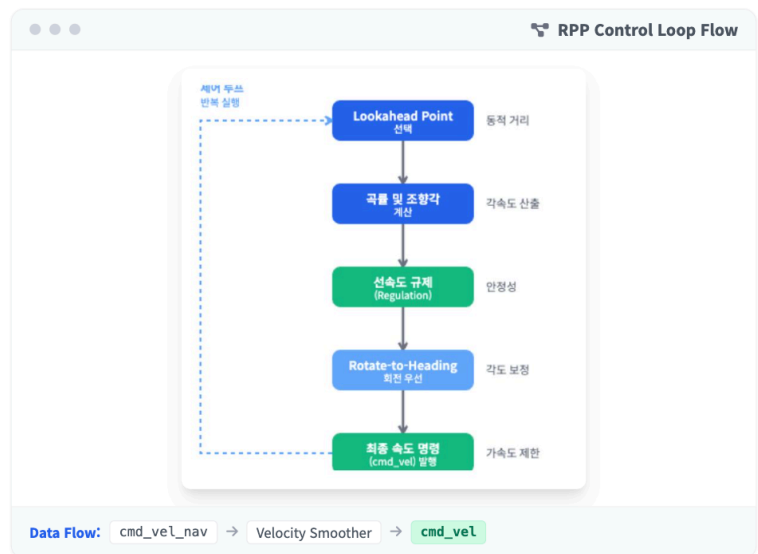
Regulated Pure Pursuit란?

핵심 제어 단계

- 1 Lookahead Point 선택**
속도 비례 동적 거리 적용 (Dynamic Lookahead)
- 2 곡률 및 조향각 계산**
원호(Arc) 가정 → 곡률(Curvature) 기반 각속도 산출
- 3 선속도 규제 (Regulation)**
급커브/장애물 근접 시 자동 감속 (안정성 확보)
- 4 Rotate-to-Heading**
헤딩 오차 큼 → 제자리 회전 우선 (각도 보정)
- 5 최종 속도 명령 발행**
가속도 제한 적용 → `cmd_vel_nav` 출력

핵심 규제 파라미터

- ☒ `use_velocity_scaled_lookahead_dist`
- ☒ `use_regulated_linear_velocity_scaling`
- ☒ `use_rotate_to_heading`



회전 우선 매커니즘

☒ `use_rotate_to_heading: true`

로봇의 현재 방향(Heading)과 이동해야 할 경로의 방향 차이가 임계값보다 클 경우, 전진하지 않고 제자리에서 회전하여 방향을 먼저 맞추는 기능입니다.

01 제자리 회전 우선

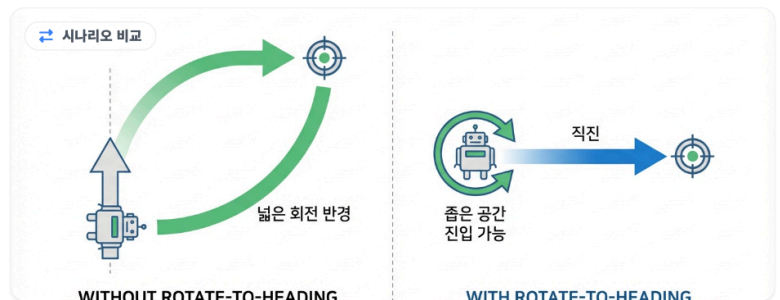
헤딩 오차가 `rotate_to_heading_min_angle` (약 20°)을 초과하면 선속도를 0으로 만들고 제자리 회전 모드로 진입합니다.

02 좁은 공간 진입 최적화

문 통과나 좁은 복도 진입 시, 넓은 회전 반경으로 인한 충돌 위험을 제거하고 정확한 각도로 안전하게 진입할 수 있게 합니다.

03 경로 이탈 복구

장애물 회피 등으로 경로를 크게 벗어난 경우, 제자리 회전을 빠르게 경로 방향을 찾아 복구 시간을 단축합니다.



nav2_params.yaml (Rotate-to-Heading)

```

FollowPath:
  plugin: "nav2_regulated_pure_pursuit_controller::RegulatedPurePursuitController"
  use_rotate_to_heading: true      # 기능 활성화
  rotate_to_heading_min_angle: 0.35 # 약 20도 이상 차이날 때 동작
  rotate_to_heading_angular_vel: 1.0 # 제자리 회전 속도 (rad/s)
  max_angular_accel: 3.2          # 빠른 반응을 위한 높은 가속도
  use_rotate_to_heading_threshold: 0.15 # 헤딩 오차가 이 값 이면 전진 전환
  allow_reversing: false          # 후진 허용 안 함 (안전)

```

회전 우선 매커니즘 설정

핵심 동작 원리

- 회전 우선 동작 (In-place Rotation)
경로 방향과 로봇 heading 차이가 클 때 전진하지 않고 제자리 회전 수행
- 회전 진입 조건
heading 오차가 rotate_to_heading_min_angle (약 20도) 이상일 때 동작
- 목표 지점 정렬 (Final Rotation)
목표 지점 도착 후 최종 방향(Yaw)을 맞추기 위한 정밀 회전 수행

Parameter Tip

max_angular_accel을 높게 설정(3.2)하여 신속하게 회전하도록 유도하며, allow_reversing은 false로 설정하여 후진 없이 회전으로만 방향을 잡습니다.

nav2_params.yaml

```
FollowPath:
  # 1) 경로 heading과 로봇 heading 차이가 나면, 먼저 in-place 회전
  use_rotate_to_heading: true
  # 2) 이 각도(rad) 이상 heading 차이가 나면 회전 모드 진입
  rotate_to_heading_min_angle: 0.35
  # 3) 회전 속도 (in-place 회전일 때 angular vel)
  rotate_to_heading_angular_vel: 1.0

  # 4) 회전할 때는 전진 속도 0에 가깝게 (거의 제자리 회전)
  max_angular_accel: 3.2
  allow_reversing: false
  use_rotate_to_heading_threshold: 0.15 # heading 오차 이내면 전진 전환

  final_rotation_alpha: 1.0
  allow_final_rotation: true
  use_final_rotation: true
```

경로 추종 관련 설정

```
70 controller_server:
71   ros__parameters:
100     FollowPath:
109       # 경로 추종 관련
110       lookahead_dist: 0.6
111       min_lookahead_dist: 0.3
112       max_lookahead_dist: 0.9
113       lookahead_time: 1.5
114       transform_tolerance: 0.2
115       use_velocity_scaled_lookahead_dist: true
116       use_collision_detection: true
117       use_interpolation: true
```

lookahead point

🎯 Lookahead Point 원리

✅ 동적 거리 조정 (Dynamic Scaling)

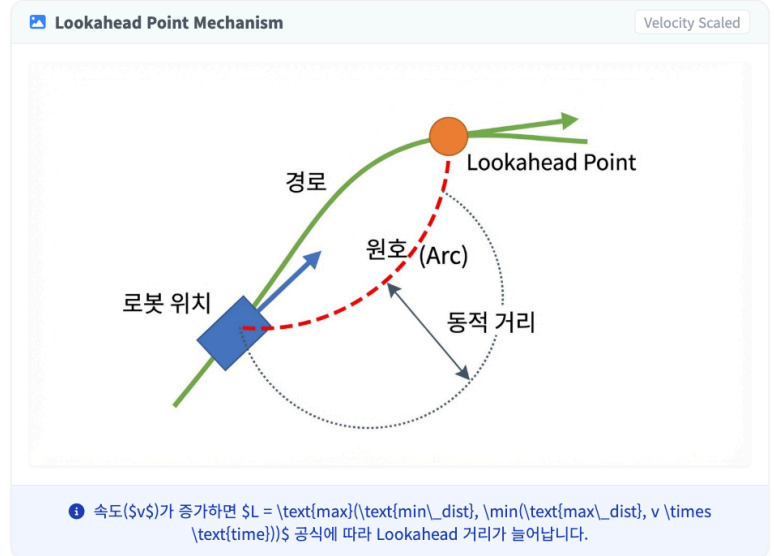
로봇의 속도가 빠를수록 Lookahead 거리를 늘려 더 멀리 보고 주행 (고속 주행 시 안정성 확보)

✅ 진동 및 오버슈트 방지

속도가 느릴 땐 짧게, 빠를 땐 길게 보아 경로 추종의 진동(Oscillation)을 최소화

🔧 핵심 파라미터

<code>use_velocity_scaled_lookahead_dist</code>	<code>true</code>
<code>lookahead_dist</code>	<code>0.6 m</code>
<code>lookahead_time</code>	<code>1.5 s</code>
<code>min/max_lookahead_dist</code>	<code>0.3 / 0.9 m</code>



Lookahead 파라미터 상세 설정

Lookahead 거리 설정

✅ 동적 Lookahead 거리

속도에 비례하여 목표 지점 거리를 조절 (`use_velocity_scaled_lookahead_dist`)

✅ 거리 제한 (Min/Max)

최소 0.3m, 최대 0.9m 범위 내에서 Lookahead 거리 유지

안전 및 보정 기능

✅ 충돌 감지 및 보간

경로 상의 장애물 감지(`use_collision_detection`) 및 부드러운 경로 보간

✅ Transform Tolerance

TF 변환 시 허용되는 시간 지연 오차 (0.2초)

```
nav2_params.yaml

FollowPath:
  # --- Lookahead 거리 설정 ---
  lookahead_dist: 0.6
  min_lookahead_dist: 0.3
  max_lookahead_dist: 0.9
  lookahead_time: 1.5
  use_velocity_scaled_lookahead_dist: true

  # --- TF 및 보간 설정 ---
  transform_tolerance: 0.2
  use_interpolation: true

  # --- 충돌 감지 ---
  use_collision_detection: true

  # (참고) 충돌 감지 시, 로컬 코스트맵 상에서
  # 경로상의 장애물을 확인하여 정지하거나 감속합니다.
```

goal 지점 근처에서 자세 조정

```
70 controller_server:
71   ros__parameters:
100     FollowPath:
135       # ===== goal 근처에서 고개 맞추기 =====
136       final_rotation_alpha: 1.0
137       allow_final_rotation: true
138       use_final_rotation: true
```

안전 관련

```

70 controller_server:
71   ros__parameters:
100     FollowPath:
140       # ===== 안전 관련 =====
141       # 장애물과 너무 가까워지면 속도 줄이거나 정지하는 옵션들
142       use_approach_velocity_scaling: true
143       min_approach_linear_velocity: 0.05
144       inflation_cost_scaling_factor: 3.0
145       regulated_linear_scaling_min_radius: 0.5
146       regulated_linear_scaling_min_speed: 0.05
147       use_cost_regulated_linear_velocity_scaling: true
148       cost_scaling_dist: 0.6
149       cost_scaling_gain: 1.5

```

local cost map

Local Costmap 호스팅 구조

✓ Controller Server 내부 실행

Controller Server는 단순히 경로를 추종하는 것뿐만 아니라, **local_costmap**을 직접 host(실행 및 관리)하여 실시간 환경 정보를 유지함

📖 Local Costmap의 역할

로봇 주변의 국소적 환경(장애물)을 고해상도로 표현하여 **실시간 충돌 방지** 및 동적 회피 수행

🌀 데이터 업데이트

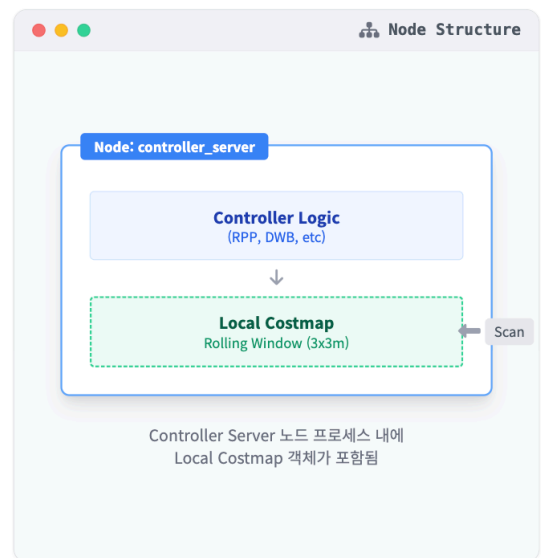
LaserScan, PointCloud 등 센서 데이터를 실시간으로 반영하여 정적 지도(Static Map)에 없는 새로운 장애물 감지

📄 참고 문서 (Official Docs)

<https://docs.nav2.org/configuration/packages/configuring-controller-server.html>

📌 핵심 포인트: 경로 추종 시 참조

Controller(FollowPath 등)는 매 제어 주기마다 Local Costmap을 참조하여, Global Planner가 생성한 경로상에 갑작스러운 장애물이 있는지 확인하고 속도를 조절하거나 정지합니다.



local cost map 설정

주요 파라미터

✓ 갱신 및 발행 주기

`update_frequency`: 맵 데이터 갱신 (5.0Hz)
`publish_frequency`: 토픽 발행 (2.0Hz)

✓ Rolling Window

로봇이 이동함에 따라 맵의 중심도 함께 이동 (Local Costmap의 특징)

✓ 맵 크기 및 해상도

3m x 3m 크기, 셀당 0.05m 해상도
 (근거리 장애물 회피에 최적화된 크기)

💡 Context: Part 1 Controller Server

Local Costmap은 Controller Server 노드 내부에서 실행(Host)되며, 실시간 장애물 회피를 위한 단기 경로 계획에 사용됩니다.

```

nav2_params.yaml

local_costmap:
  local_costmap:
    ros__parameters:
      # 맵 데이터 갱신 주기 (Hz)
      update_frequency: 5.0
      # 시각화 토픽 발행 주기 (Hz)
      publish_frequency: 2.0

      # 로봇 중심으로 맵 윈도우 이동
      rolling_window: true

      # 맵 크기 (미터 단위)
      width: 3
      height: 3

      # 맵 해상도 (미터/셀)
      resolution: 0.05

```

좌표계 설정

Part 1: Controller Server Local Costmap

좌표계 설정 (TF Frames)

- global_frame: odom**
로컬 맵의 기준 좌표계. 로봇 중심의 상대적 이동을 반영하며 map보다 갱신이 빠름
- robot_base_frame: base_footprint**
로봇의 중심 좌표계. 지면 투영 기준점

로봇 풋프린트 (Footprint)

- footprint 정의**
단순 반경(robot_radius) 대신 4개 꼭짓점 좌표로 정확한 형상 정의 (충돌 계산 정밀도 향상)

```
nav2_params.yaml

local_costmap:
  local_costmap:
    ros__parameters:
      # --- 좌표계 설정 ---
      global_frame: odom          # 로컬 맵 기준 (map 아님)
      robot_base_frame: base_footprint

      # --- 로봇 형상 정의 ---
      # robot_radius: 0.22 # 단순 원형 모델 (비활성)

      footprint: '[[0.06, 0.06], [0.06, -0.06], [-0.06, -0.06], [-0.06, 0.06]]'
      # 1. 전방 좌측 [0.06, 0.06]
      # 2. 전방 우측 [0.06, -0.06]
      # 3. 후방 우측 [-0.06, -0.06]
      # 4. 후방 좌측 [-0.06, 0.06]
```

layer 플러그인

CONTEXT Part 1: Controller Server (Local Costmap Host)

주요 레이어 구성

- 플러그인 리스트 (plugins)**
Costmap을 구성할 레이어들의 실행 순서 정의 (순차적 처리)
- Voxel Layer (3D 장애물)**
센서 데이터를 기반으로 3D 공간상의 장애물을 감지하고 점유/클리어 상태 반영
- Inflation Layer (비용 확장)**
장애물 주변에 안전 비용(Cost)을 부여하여 충돌 방지 및 안전 거리 확보
- 확장성 (Extensibility)**
필요 시 static_layer 등을 추가하여 기능 확장 가능

```
nav2_params.yaml

local_costmap:
  local_costmap:
    ros__parameters:
      # 사용할 레이어 플러그인 목록 정의 (실행 순서 중요)
      plugins: ["voxel_layer", "inflation_layer"]

      # 1. Voxel Layer: 3D 장애물 감지 및 맵핑
      voxel_layer:
        plugin: "nav2_costmap_2d::VoxelLayer"
        enabled: True
        # ... (세부 설정은 다음 페이지에서 계속)

      # 2. Inflation Layer: 장애물 주변 비용 확장
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        cost_scaling_factor: 3.0
        inflation_radius: 0.15
```

inflation layer

주요 파라미터

- ✔ **plugin**
`nav2_costmap_2d::InflationLayer` 사용
- ✔ **cost_scaling_factor**
장애물로부터 멀어질수록 비용이 감소하는 비율 (값이 클수록 급격히 감소)
기본값: 3.0 (장애물 근처 회피 경향 강화)
- ✔ **inflation_radius**
장애물 주변에 비용이 부여되는 최대 반경 (안전 여유 공간)
설정값: 0.15m (로봇 크기 고려하여 설정)

❗ 작동 원리

Inflation Layer는 장애물(Lethal Cost) 주변 셀에 지수 함수적으로 감소하는 비용을 할당하여, 경로 계획 시 로봇이 장애물에 너무 근접하지 않도록 '가상 벽' 역할을 수행합니다.

```
local_costmap:
  local_costmap:
    ros__parameters:
      # ... (이전 설정 생략)

  inflation_layer:
    plugin: "nav2_costmap_2d::InflationLayer"

    # 비용 감소 계수 (값이 클수록 장애물 근처 비용 급감)
    cost_scaling_factor: 3.0

    # 인플레이션 반경 (비용이 적용되는 최대 거리)
    inflation_radius: 0.15
```

static layer

Static Layer 설정

- ✔ **정적 지도 레이어 정의**
사전 생성된 지도(SLAM 결과)를 Costmap에 반영
- ✔ **현재 상태: 비활성**
Local Costmap은 주로 동적 장애물 회피 목적이므로 기본적으로 코멘트 처리됨
- ✔ **Transient Local 구독**
지도를 늦게 구독하더라도 이전 메시지를 수신하도록 QoS 설정
(`map_subscribe_transient_local: True`)

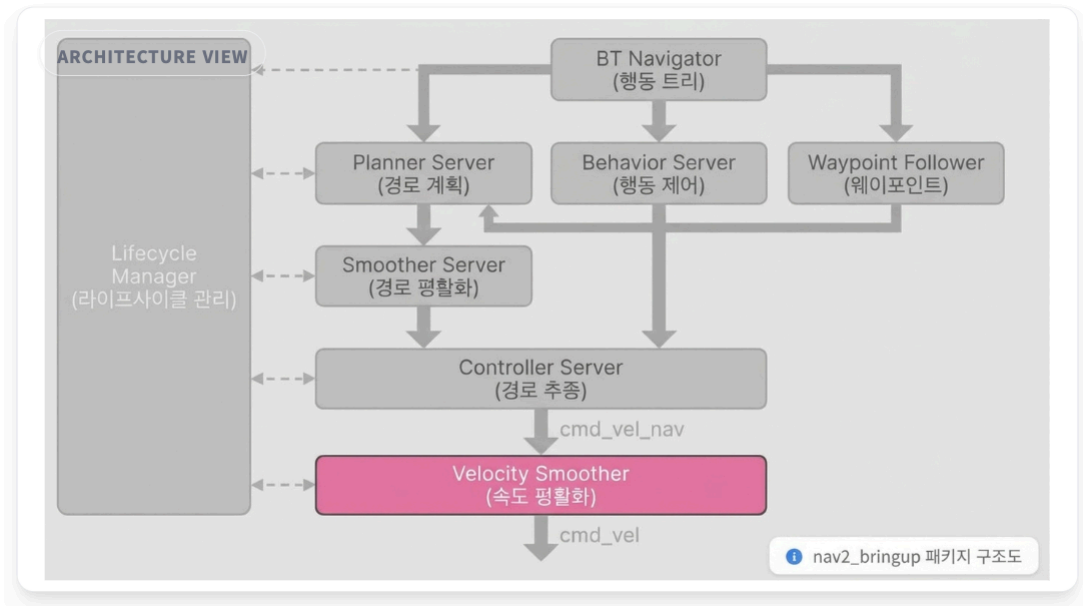
전체 맵 전송 옵션

- ✔ **always_send_full_costmap**
`True` 설정 시, 업데이트된 부분만 보내지 않고 전체 맵 데이터를 매번 전송
- ✔ **사용 목적**
네트워크 대역폭을 더 사용하지만, 수신 측(Rviz 등)의 데이터 동기화 안정성 향상

```
local_costmap:
  local_costmap:
    ros__parameters:
      # 정적 레이어 (필요 시 plugins에 추가하여 활성화)
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer"
        map_subscribe_transient_local: True

      # 전체 맵 전송 설정
      always_send_full_costmap: True
```

Velocity Smoother



속도 평활화 효과

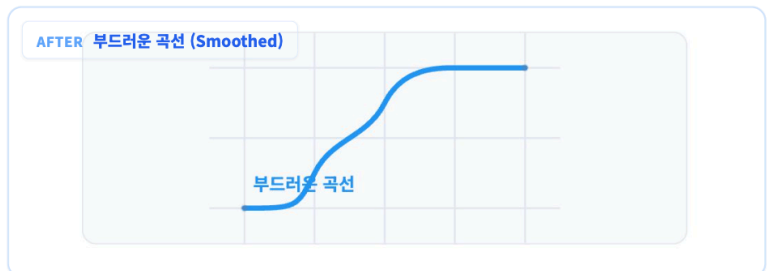
❌ Smoother 미적용 (Raw Input)

- **급격한 속도 변화:** 목표 속도로 즉시 점프(Step Input)하여 각진 그래프 형성
- **기계적 충격:** 모터와 기어박스에 순간적인 과부하 전달
- 로봇의 거동이 불안정하고 덜컥거림(Jerk) 발생



✅ Smoother 적용 (Smoothed Output)

- **부드러운 가속속:** 설정된 가속도 한계 내에서 완만한 곡선 생성
- **승차감 향상:** 물류 이송 시 적재물 쏠림 방지 및 안정적 주행
- 하드웨어 수명 연장 및 배터리 효율 증대



velocity smoother 실행

주요 구성 요소

Composable Node 실행

`nav2_velocity_smoother` 패키지의 `VelocitySmoother` 플러그인 로드

입력 리맵핑 (Input Remap)

Controller Server의 출력인 `cmd_vel_nav` 를 `cmd_vel` 로 받아 입력 처리

출력 리맵핑 (Output Remap)

평활화된 명령 `cmd_vel_smoothed` 를 최종 로봇 구동 토픽인 `cmd_vel` 로 리맵핑

공통 파라미터 적용

전체 파라미터 파일 로딩 및 시뮬레이션 시간 동기화 설정

```
navigation_launch.xml

<composable_node pkg="nav2_velocity_smoother"
  plugin="nav2_velocity_smoother::VelocitySmoother"
  name="velocity_smoother">

  <param from="$(var params_file)"/>
  <param name="use_sim_time" value="$(var use_sim_time)"/>
  <param name="autostart" value="$(var autostart)"/>

  <remap from="/tf" to="tf"/> <remap from="/tf_static" to="tf_static"/>

  <!-- 입력: Controller Server 출력 (cmd_vel_nav) -->
  <remap from="cmd_vel" to="cmd_vel_nav"/>

  <!-- 출력: 최종 로봇 구동 명령 (cmd_vel) -->
  <remap from="cmd_vel_smoothed" to="cmd_vel"/>

</composable_node>
```

속도 제한 설정

주요 파라미터 (Parameters)

max_velocity

[x, y, theta] 형식의 최대 속도 제한

x: 0.25 m/s theta: 1.5 rad/s

min_velocity

후진 및 반대 회전 시 최소 속도 제한

x: -0.25 m/s theta: -1.5 rad/s

차동구동 로봇 특성 (Differential Drive)

Y축 속도(회이동)는 구조적으로 불가능하므로 0.0으로 설정하여 측면 이동 명령 무시

Safety Safety

과도한 속도 명령을 사전에 차단(Clamping)하여 하드웨어를 보호하고 안전한 자율주행을 보장합니다.

```
nav2_params.yaml

velocity_smoother:
  ros_parameters:
    # 속도 제한 설정 [x, y, theta]
    # x: 전진 속도 (m/s)
    # y: 횡이동 속도 (m/s) - 차동구동 로봇은 0.0
    # theta: 회전 속도 (rad/s)

    max_velocity: [0.25, 0.0, 1.5]

    # 최소 속도 (음수값은 후진 및 시계방향 회전)
    min_velocity: [-0.25, 0.0, -1.5]

    # 참고:
    # Omni/Mecanum 로봇의 경우 y축 값 설정 필요
    # 예: max_velocity: [0.25, 0.25, 1.5]
```

가속도 제한 설정

주요 파라미터 (Parameters)

- ✔ **최대 가속도 (max_accel)**
값: [2.5, 0.0, 3.2] (x, y, theta)
로봇의 급출발을 방지하고 부드러운 출발 유도
- ✔ **최대 감속도 (max_decel)**
값: [-2.5, 0.0, -3.2] (음수 값)
급정지 시 바퀴 미끄러짐 방지 및 기계적 충격 완화
- ✔ **부드러운 가감속 보장**
급격한 속도 변화 명령을 시간 기반 램핑(Ramping)으로 변환하여 하드웨어 보호 및 승차감 향상

```
nav2_params.yaml

velocity_smoother:
  ros_parameters:
    # 최대 가속도 설정 [x_accel, y_accel, theta_accel]
    # x: 전진, y: 횡이동(0.0), theta: 회전
    max_accel: [2.5, 0.0, 3.2]

    # 최대 감속도 설정 (음수 값으로 지정)
    # x: 전진 감속, y: 0.0, theta: 회전 감속
    max_decel: [-2.5, 0.0, -3.2]

    # 데드밴드 (노이즈 제거용 최소 속도)
    deadband_velocity: [0.0, 0.0, 0.0]

    # 명령 타임아웃 (통신 두절 시 정지 시간)
    velocity_timeout: 1.0
```

Odometry 및 타임아웃

오도메트리 (Odometry)

- 📡 **odom_topic**
로봇의 현재 속도 정보를 수신할 토픽 이름 지정 (기본값: "odom")
- 🕒 **odom_duration**
수신된 오도메트리 데이터의 유효 시간 윈도우 (0.1초)

타임아웃 (Timeout)

- 🔴 **velocity_timeout**
입력 명령(cmd_vel_nav)이 끊길 경우 로봇을 정지시키는 시간 임계값 (1.0 초)
- 📘 **OPEN_LOOP 특성**
오도메트리 피드백 의존도가 낮으며, 주로 시간 기반 램핑(Ramping) 처리에 집중

```
nav2_params.yaml

velocity_smoother:
  ros_parameters:
    # 오도메트리 토픽 및 유효 시간 설정
    odom_topic: "odom"
    odom_duration: 0.1

    # 입력 명령 타임아웃 (안전 장치)
    # 1초 동안 새로운 cmd_vel_nav가 없으면 정지
    velocity_timeout: 1.0

    # 참고: OPEN_LOOP 모드에서는 실제 속도 피드백보다
    # 명령 값 기반의 스무딩 처리가 우선됩니다.
    feedback: "OPEN_LOOP"
```

Deadband

Deadband 설정

- ✓ **deadband_velocity**
해당 값 이하의 미세 속도 명령은 0.0으로 무시
예: [0.0, 0.0, 0.0] → 모든 미세 입력 허용

→ 저속 떨림 억제
조이스틱 중립 오차나 노이즈로 인한 미세 움직임 제거

메시지 형식 설정

- ✓ **enable_stamped_cmd_vel**
false: geometry_msgs/Twist 사용
true: TwistStamped (헤더 포함) 사용

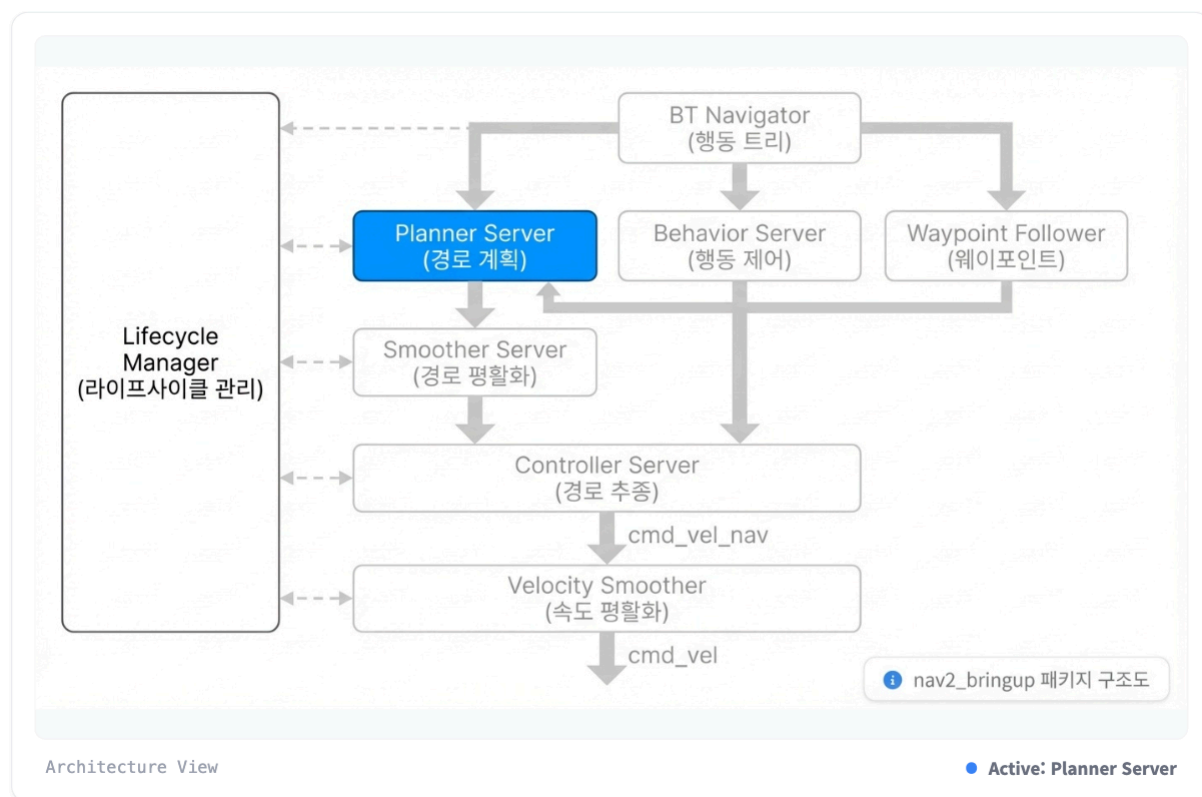
```
nav2_params.yaml

velocity_smoother:
  ros__parameters:
    # ... (이전 설정: 오도메트리 등) ...

    # 불감대(Deadband) 설정: [x, y, theta]
    # 이 값보다 작은 속도 명령은 0으로 처리하여 노이즈 제거
    deadband_velocity: [0.0, 0.0, 0.0]

    # 메시지 타입 설정
    # False: geometry_msgs/Twist (헤더 없음, 일반적)
    # True: geometry_msgs/TwistStamped (헤더 포함)
    enable_stamped_cmd_vel: false
```

Planner server



Planner Server 실행

주요 실행 설정

패키지 및 플러그인

`nav2_planner` 패키지의 `PlannerServer` 플러그인을 사용하여 노드 실행

파라미터 로딩

`params_file` 변수로부터 `nav2_params.yaml` 설정을 로드하여 적용

TF 리맵핑 (Remap)

시스템 호환성을 위해 `/tf` 및 `/tf_static` 토픽을 상대 경로로 리맵핑

공통 실행 인자

시뮬레이션 시간 동기화(`use_sim_time`) 및 자동 시작(`autostart`) 옵션 적용

```
<composable_node pkg="nav2_planner"
  plugin="nav2_planner::PlannerServer"
  name="planner_server">

  <!-- 파라미터 파일 로딩 -->
  <param from="$(var params_file)"/>

  <!-- 공통 설정 적용 -->
  <param name="use_sim_time" value="$(var use_sim_time)"/>
  <param name="autostart" value="$(var autostart)"/>

  <!-- TF 리맵핑 -->
  <remap from="/tf" to="tf"/>
  <remap from="/tf_static" to="tf_static"/>
</composable_node>
```

경로 계획 주기

경로 계획 주기 (Frequency)

✓ expected_planner_frequency: 20.0Hz

경로 계획 서버의 목표 실행 주기 (초당 20회)

ⓘ 주의사항

목표 주기일 뿐이며, 시스템 부하에 따라 실제 실행 주기는 달라질 수 있음

코스트맵 업데이트 타임아웃

✓ costmap_update_timeout: 1.0s

Global Costmap 최신화 지연 시 대기 한계 시간

⚠ 안전 메커니즘

코스트맵이 최신 상태가 아니면 경로 생성을 지연하거나 실패 처리하여 안전 확보

```
planner_server:
  ros_parameters:
    # 플래너 호출 목표 주기 (Hz)
    expected_planner_frequency: 20.0

    # Costmap 업데이트 대기 타임아웃 (초)
    costmap_update_timeout: 1.0

    # 사용할 플래너 플러그인 목록 (다음 페이지 참조)
    planner_plugins: ["GridBased"]
```

NavFn 파라미터

🎯 tolerance: 0.5

목표 지점 근접 허용 오차(m). 정확한 목표 지점이 장애물 내부에 있거나 도달 불가능할 경우, 이 반경 내에서 가장 가까운 유효한 지점을 목표로 설정합니다.

🔗 use_astar: false

경로 계획 알고리즘 선택 옵션. `False` 일 경우 Dijkstra(다익스트라) 알고리즘을 사용하여 전체 맵을 탐색하며 최단 경로를 보장합니다.

👤 allow_unknown: true

미지도(Unknown) 영역 통과 허용 여부. 초기 탐사나 지도가 불완전한 환경에서 로봇이 미지의 공간을 가로질러 경로를 생성할 수 있도록 합니다.

💡 Tuning Tip

실내 지도 품질이 낮거나 복잡한 환경에서는 `tolerance`를 늘려 경로 실패를 줄이고, 대형 맵에서는 `use_astar: true`로 변경하여 연산 속도를 높이는 것을 고려하세요.

```
nav2_params.yaml

planner_server:
  ros_parameters:
    expected_planner_frequency: 20.0
    planner_plugins: ["GridBased"]

    # GridBased 플러그인 상세 설정
    GridBased:
      plugin: "nav2_navfn_planner::NavfnPlanner"

      # 목표 지점 근접 허용 오차 (m)
      tolerance: 0.5

      # A* 대신 Dijkstra 사용 (False)
      use_astar: false

      # 미지도 영역(Unknown space) 통과 허용
      allow_unknown: true
```

Global Cost map

핵심 역할 및 구조

🖥️ 호스팅 주체 (Host)

Global Costmap은 **Planner Server** 노드 내부에서 직접 실행되며 관리됩니다.

🌐 전역 경로 계획 (Global Planning)

전체 정적 지도(Static Map)를 기반으로 하여 시작점부터 목표점까지의 최적 경로를 계산하는 데 사용됩니다.

🏗️ 계층 구조 및 비교

Global: 전체 맵 기준, 낮은 빈도 업데이트
Local: 로봇 주변 기준, 높은 빈도 (Controller Server)

```
nav2_params.yaml (Global Costmap Context)

global_costmap:
  ros_parameters:
    # 지도 업데이트 및 발행 빈도 (Local보다 낮음)
    update_frequency: 1.0 ... publish_frequency: 1.0
    # 전역 좌표계 설정
    global_frame: map ... robot_base_frame: base_footprint
    # 로봇 크기 정의
    robot_radius: 0.22 ... resolution: 0.05 ... track_unknown_space: True
    # 레이어 플러그인 (정적 지도 + 장애물 + 인플레이션)
    plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
    static_layer:
      plugin: "nav2_costmap_2d::StaticLayer"
      map_subscribe_transient_local: True
    inflation_layer:
      plugin: "nav2_costmap_2d::InflationLayer"
      cost_scaling_factor: 3.0
      inflation_radius: 0.55
```

Dijkstra와 A*의 차이점 및 use_astar 스위치

특징	Dijkstra (NavFn 기본)	A* (A-Star)
탐색 방식	모든 방향 균등 탐색 (BFS 유사)	목표 지향적 탐색 (Heuristic)
계산 효율	낮음 (전체 맵 스캔)	높음 (필요 영역만 탐색)
경로 품질	최단 경로 보장 (Optimal)	휴리스틱에 따라 달라짐
적합 환경	복잡한 미로, 소형 맵	개방된 공간, 대형 맵

- use_astar** 파라미터

`false` 설정 시 Dijkstra 알고리즘을 사용하며(기본값), `true` 로 설정하면 휴리스틱을 사용하는 A* 알고리즘으로 전환됩니다.
- 선택 가이드**

실내 환경과 같이 맵이 크지 않고 최적 경로가 중요한 경우 **Dijkstra**가 유리하며, 실외나 대형 물류 창고처럼 맵이 매우 크고 빠른 응답성이 필요한 경우 **A*** 사용을 권장합니다.

nav2_params.yaml

```

planner_server:
  ros_parameters:
    GridBased:
      # use_astar: false -> Dijkstra (기본값)
      # use_astar: true  -> A* (A-Star)

      use_astar: true

# Dijkstra:
# - 모든 방향으로 균등하게 탐색
# - 항상 최단 경로 보장
# - 계산 비용 높음

# A* (A-Star):
# - 목표 방향으로 우선 탐색 (Heuristic)
# - 탐색 속도 빠름
# - 휴리스틱 함수에 따라 경로 품질 변화

```

글로벌 코스트 맵

주요 개념 및 역할

- 호스팅 구조 (Hosting)**

Global Costmap은 별도의 노드가 아니라, **Planner Server 노드 내부**에서 객체 형태로 생성 및 관리됩니다.
- 전역 계획 (Global Planning)**

전체 지도와 정적 장애물 정보를 기반으로 시작점에서 목표점까지의 최적 경로를 계산할 때 사용됩니다.
- Local Costmap과의 차이**

Local Costmap은 Controller Server가 호스팅하며, 실시간 동적 장애물 회피에 중점을 둡니다.

공식 문서 참조

자세한 설정 방법은 아래 공식 문서를 확인하세요.

<https://docs.nav2.org/configuration/packages/configu...>

nav2_params.yaml (구조 예시)

```

# Planner Server 설정 영역
planner_server:
  ros_parameters:
    expected_planner_frequency: 20.0
    planner_plugins: ["GridBased"]
    # ... 기타 플래너 설정 ...

# Global Costmap 설정 영역
# Planner Server가 이 설정을 로드하여 Costmap 객체를 생성합니다.
global_costmap:
  global_costmap:
    ros_parameters:
      update_frequency: 1.0
      publish_frequency: 1.0
      global_frame: map
      robot_base_frame: base_footprint

```

Global Costmap 플러그인 레이어 구조



1. Static Layer (정적 지도)

입력: Map Server (OccupancyGrid)

변하지 않는 벽, 기둥 등 사전에 작성된 지도 정보를 비용으로 변환하여 기본 골격을 형성합니다.



2. Obstacle Layer (동적 장애물)

입력: LiDAR, Depth Camera (Scan)

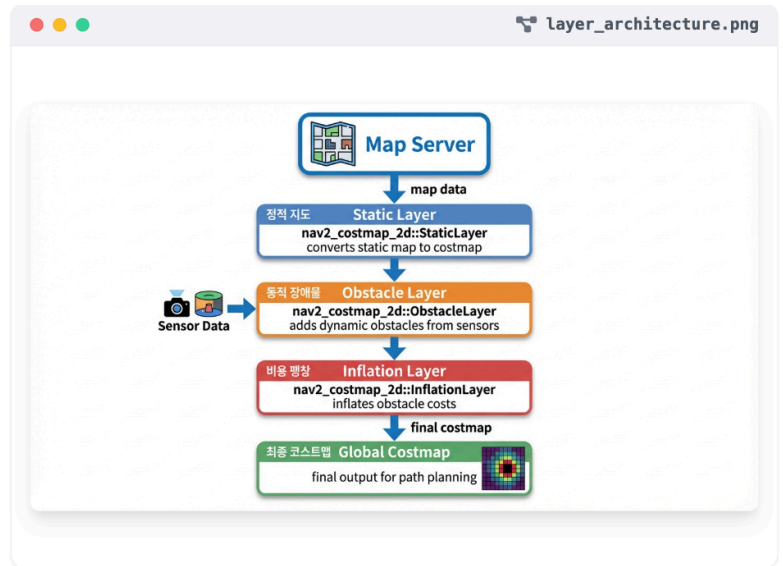
실시간 센서 데이터를 반영하여 이동하는 사람이나 새로운 물체를 장애물 비용으로 추가/삭제합니다.



3. Inflation Layer (비용 팽창)

입력: 합성된 Costmap

장애물 주변에 안전 마진(Buffer)을 생성하여, 로봇이 벽에 너무 가까이 충돌하지 않도록 경로를 유도합니다.



좌표계 설정

주요 파라미터 (Key Parameters)

Robot Footprint (형상)

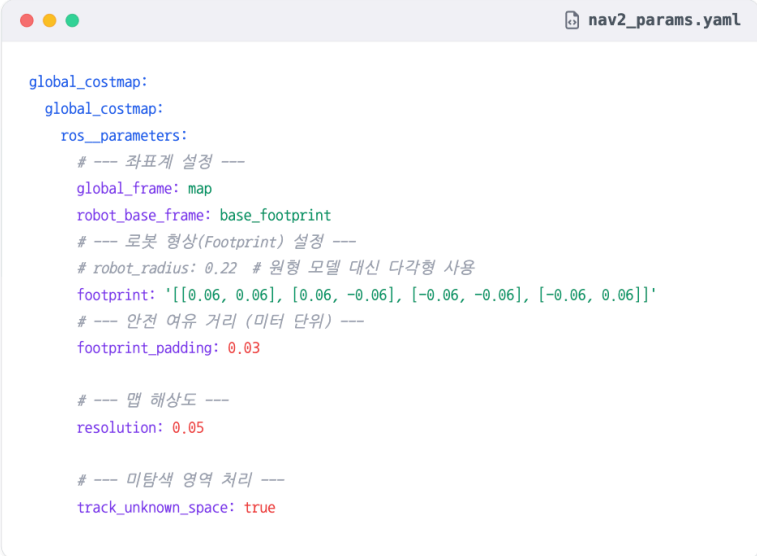
단순 반경(radius) 대신 다각형 좌표로 로봇의 실제 형상을 정밀하게 정의합니다. 이는 좁은 공간 통과 능력을 향상시킵니다.

Footprint Padding

로봇 형상 외부에 0.03m의 추가 여유 공간을 두어, 물리적 충돌을 방지하고 보수적인 경로 계획을 유도합니다.

Global Frame & Robot Base

비용 지도는 map 프레임(고정)을 기준으로 생성되며, 로봇의 위치는 base_footprint를 기준으로 투영됩니다.



플러그인 구조 개요

주요 레이어 구성

- Static Layer (정적 레이어)**
SLAM 등으로 생성된 정적 지도를 기반으로 벽이나 고정 장애물 정보를 비용 지도에 반영합니다.
- Obstacle Layer (장애물 레이어)**
Lidar, Depth Camera 등 실시간 센서 데이터를 통해 움직이는 사람이나 새로운 물체를 감지합니다.
- Inflation Layer (팽창 레이어)**
장애물 주변에 비용을 점진적으로 감소시키며 확장하여 로봇이 장애물과 안전 거리를 유지하도록 유도합니다.

Tip: 플러그인 리스트에 정의된 순서대로 레이어가 적용되어 최종 비용 지도가 계산됩니다.

```
nav2_params.yaml

global_costmap:
  # 플러그인 로딩 순서 정의 (순서 중요)
  plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
  # 1. 정적 지도 레이어 정의
  static_layer:
    plugin: "nav2_costmap_2d::StaticLayer"
    map_subscribe_transient_local: True
  # 2. 동적 장애물 레이어 정의
  obstacle_layer:
    plugin: "nav2_costmap_2d::ObstacleLayer"
    enabled: True
    observation_sources: scan
  # 3. 비용 팽창 레이어 정의
  inflation_layer:
    plugin: "nav2_costmap_2d::InflationLayer"
    cost_scaling_factor: 3.0
    inflation_radius: 0.55
```

Static Layer 상세 설정

주요 파라미터

- ✓ **plugin: StaticLayer**
Map Server가 제공하는 정적 지도(OccupancyGrid)를 구독하여 비용 지도의 기초 레이어로 사용합니다.
- ✓ **map_subscribe_transient_local**
지도 토픽의 QoS를 `Transient Local`로 설정합니다.
지도는 보통 1회만 발행되므로, 뒤늦게 실행된 노드도 데이터를 받을 수 있도록 이 설정이 필수입니다 (Latched 개념).
- ✓ **전역 계획의 기초**
벽, 기둥 등 고정된 장애물 정보를 제공하여 전역 경로 계획의 기본 골격을 형성합니다.

```
nav2_params.yaml

# Planner Server의 Global Costmap 설정
global_costmap:
  # 레이어 플러그인 목록 (순서 중요)
  plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
  # Static Layer 상세 설정
  static_layer:
    plugin: "nav2_costmap_2d::StaticLayer"
    # 매우 중요: 지도 토픽 QoS 설정
    # True: Transient Local (Latched) - 늦게 접속해도 지도 수신
    # False: Volatile - 발행 순간에만 수신 가능 (지도 수신 실패 원인)
    map_subscribe_transient_local: True

    # (Optional) 사용할 지도 토픽 이름
    # map_topic: "/map"

    # (Optional) Unknown 공간 처리
    # subscribe_to_updates: True
```

Obstacle Layer 상세: 센서 통합

주요 파라미터 (Scan Source)

- Observation Sources & Topic**
`scan` 소스 정의 및 2D LiDAR 토픽(`/scan`) 연결. 실시간 장애물 정보를 수신합니다.
- Clearing vs Marking**
Clearing: 센서와 장애물 사이 공간을 비움(이동 가능).
Marking: 장애물이 감지된 지점을 점유(이동 불가)로 표시.
- Raytrace & Obstacle Range**
raytrace: 이 거리까지 빈 공간으로 처리 (3.0m).
obstacle: 이 거리 이내 장애물만 지도에 표시 (2.5m).

```
global_costmap:
  global_costmap:
    ros__parameters:
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        enabled: True
        observation_sources: scan
      scan:
        topic: /scan
        max_obstacle_height: 2.0
        clearing: True # 장애물 사이 공간을 비움
        marking: True # 장애물을 지도에 기록
        data_type: "LaserScan"
        raytrace_max_range: 3.0
        raytrace_min_range: 0.0
        obstacle_max_range: 2.5
        obstacle_min_range: 0.0
```

Inflation Layer 상세 및 기타 설정

Inflation Layer (비용 팽창)

- cost_scaling_factor: 3.0**
비용 감소 기술기입니다. 값이 클수록 장애물에서 멀어질 때 비용이 급격히 감소하여, 로봇이 장애물에 더 가까이 접근할 수 있게 됩니다.
- inflation_radius: 0.15**
장애물 주변의 팽창 반경(m)입니다. 로봇의 반지름을 고려하여 설정하며, 이 영역 안으로는 경로가 생성되지 않도록 유도합니다.

기타 설정

- always_send_full_costmap**
`True`로 설정 시 매 주기마다 전체 맵 데이터를 발행합니다. 네트워크 부하는 증가하지만, 클라이언트(RViz)에서의 맵 동기화 문제를 방지합니다.

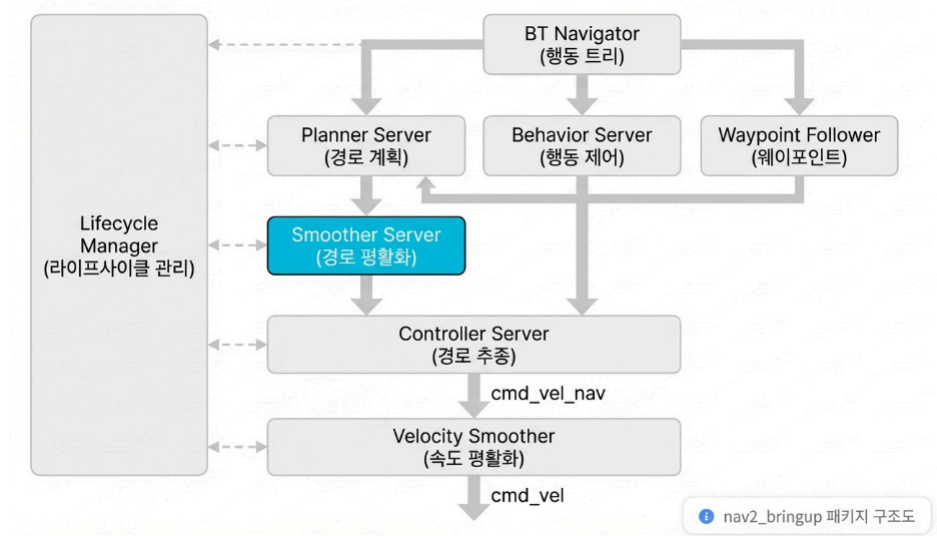
```
global_costmap:
  global_costmap:
    ros__parameters:
      # Inflation Layer 설정
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        cost_scaling_factor: 3.0 # 비용 감소 기술기
        inflation_radius: 0.15 # 팽창 반경 (m)

      # 기타 설정
      always_send_full_costmap: True

      # (참고) 이전 페이지 설정 요약
      # plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
      # static_layer: ...
      # obstacle_layer: ...
```

Smoother Server

구조



Smoother Server 노드 실행 분석

노드 개요 및 역할

경로 후처리 (Path Smoothing)

GridSearch(A* 등)로 생성된 거친 경로를 부드럽게 보정하여 로봇의 주행 안정성과 승차감을 향상시킵니다.

Nav2 Smoother 패키지

Lifecycle Node 기반의 `nav2_smoother::SmootherServer`를 플러그인 형태로 로드하여 실행합니다.

파이프라인 위치

Planner Server(전역 계획) 실행 후, Controller Server(추종) 실행 전 단계에서 경로를 최적화합니다.

TF Remapping

네임스페이스 충돌 방지를 위해 글로벌 토픽 `/tf`를 로컬 `tf`로 리매핑하여 격리된 환경에서도 좌표 변환이 올바르게 동작하도록 설정합니다.

```
navigation_launch.xml

<!-- Smoother Server: 경로 스무딩 노드 -->
<composable_node pkg="nav2_smoother"
  plugin="nav2_smoother::SmootherServer"
  name="smoother_server">

  <!-- 파라미터 파일 로딩 -->
  <param from="$(var params_file)"/>

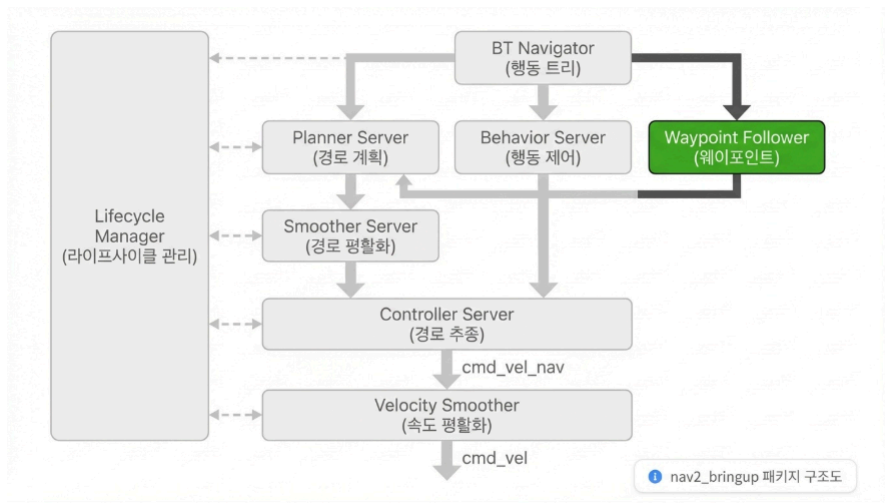
  <!-- 공통 설정 적용 -->
  <param name="use_sim_time" value="$(var use_sim_time)"/>
  <param name="autostart" value="$(var autostart)"/>

  <!-- TF 리매핑 (네임스페이스 호환) -->
  <remap from="/tf" to="tf"/>
  <remap from="/tf_static" to="tf_static"/>

</composable_node>
```

Waypoint Follower

구조



Waypoint Follower 노드 실행 분석

주요 구성 요소

목적 및 역할

지정된 웨이포인트 목록을 순차적으로 방문하며, 도착 시마다 정의된 태스크 (Task)를 실행하여 복합 미션을 수행합니다.

노드 구성 정보

nav2_waypoint_follower 패키지의 WaypointFollower 플러그인을 사용하여 Lifecycle 노드로 실행됩니다.

파라미터 및 호환성

params_file 을 통해 설정을 로딩하고, /tf 리맵핑으로 네임스페이스 격리 환경을 지원합니다.

```

navigation_launch.xml

<composable_node pkg="nav2_waypoint_follower"
  plugin="nav2_waypoint_follower::WaypointFollower"
  name="waypoint_follower">

  <!-- 파라미터 파일 로딩 -->
  <param from="$(var params_file)"/>
  <param name="use_sim_time" value="$(var use_sim_time)"/>
  <param name="autostart" value="$(var autostart)"/>

  <!-- TF 리맵핑 (네임스페이스 지원) -->
  <remap from="/tf" to="tf"/>
  <remap from="/tf_static" to="tf_static"/>

</composable_node>

```

Waypoint Follower 기본 동작 설정



loop_rate (20Hz)

내부 제어 루프의 실행 주기입니다. 액션 서버의 상태를 확인하고 피드백을 발행하는 빈도를 결정하며, 너무 낮으면 반응성이 떨어질 수 있습니다.



stop_on_failure (False)

특정 웨이포인트 태스크 실패 시 전체 미션 중단 여부입니다. `false` 로 설정 시 실패한 지점을 건너뛰고 다음 목표로 계속 진행합니다.



action_server_result_timeout (900.0s)

각 웨이포인트 도착 후 실행되는 태스크(플러그인)의 완료를 기다리는 최대 시간입니다. 긴 작업(충전, 정밀 스캔 등)이 필요한 경우 충분히 늘려야 합니다.

```
nav2_params.yaml

waypoint_follower:
  # 루프 실행 주기 (Hz)
  loop_rate: 20

  # 웨이포인트 실행 실패 시 중단 여부
  # false: 실패해도 다음 웨이포인트로 진행 (미션 연속성)
  # true: 즉시 미션 전체 취소
  stop_on_failure: false

  # 태스크 액션 서버 결과 대기 타임아웃 (초)
  action_server_result_timeout: 900.0

  # 웨이포인트 실행자 플러그인 지정
  waypoint_task_executor_plugin: "wait_at_waypoint"

  # 플러그인별 상세 설정은 하위에 기술...
```

플러그인 구조: waypoint_task_executor_plugin

구조 및 설정 규칙



플러그인 기반 태스크 실행

도착 지점마다 수행할 동작(사진 촬영, 대기, 로봇 팔 조작 등)을 플러그인 형태로 정의하여 유연하게 교체 가능합니다.



플러그인 선택 (Executor)

`waypoint_task_executor_plugin` 파라미터에 사용할 플러그인의 이름을 문자열로 지정합니다.



네임스페이스 계층 구조

선택한 플러그인 이름(`wait_at_waypoint`) 하위에 실제 클래스 경로(`plugin`)와 전용 파라미터를 배치해야 합니다.

```
nav2_params.yaml

waypoint_follower:
  ros_parameters:
    # 사용할 태스크 플러그인 이름 지정
    waypoint_task_executor_plugin: "wait_at_waypoint"

    # 플러그인 별 상세 설정 (이름이 위와 일치해야 함)
    wait_at_waypoint:
      plugin: "nav2_waypoint_follower::WaitAtWaypoint"
      enabled: True
      waypoint_pause_duration: 200 # 대기 시간 설정
```

WaitAtWaypoint 플러그인 상세

주요 파라미터 (Parameters)

- plugin**
`nav2_waypoint_follower::WaitAtWaypoint` 클래스를 로드하여 도착 지점에서
의 정지 로직을 처리합니다.
- enabled**
플러그인 활성화 여부를 설정합니다. `True` 일 경우 웨이포인트마다 대기 동
작이 실행됩니다.
- waypoint_pause_duration**
웨이포인트 도착 후 대기할 시간을 설정합니다. 값 200은 시스템 루프 주기에
따른 틱(Tick) 수 또는 밀리초(ms) 단위로 해석될 수 있습니다.
- 활용 사례**
보안 순찰 시 사진 촬영을 위한 정지, 재고 조사 스캔, 또는 다음 명령 수신 대
기 등에 활용됩니다.

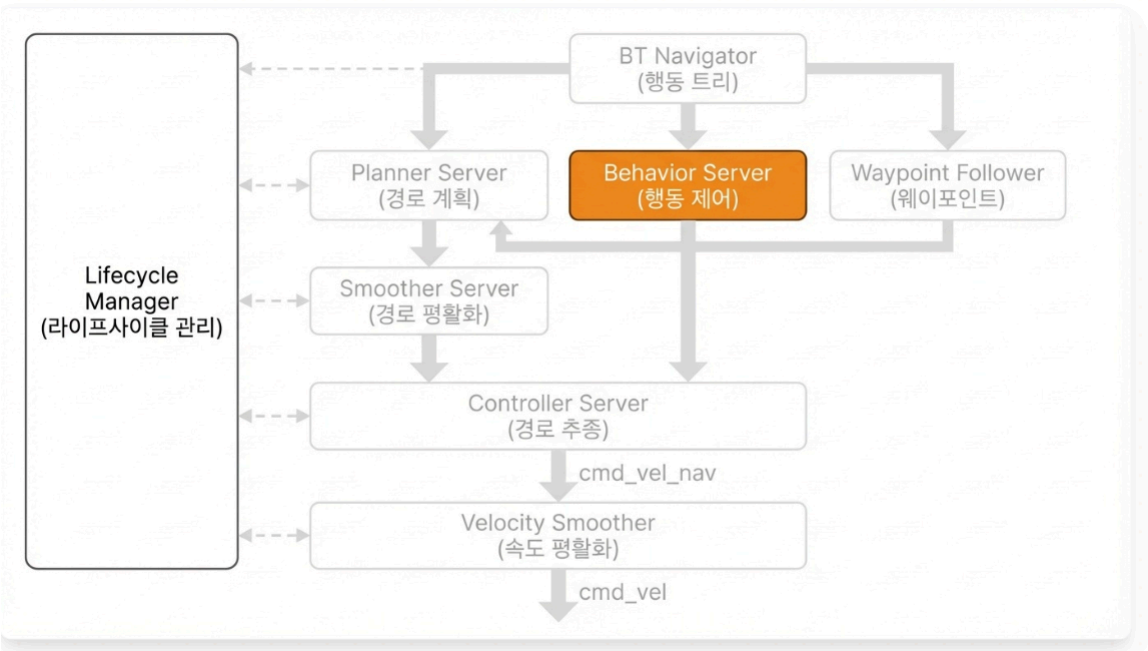
```
nav2_params.yaml

# Waypoint Follower Configuration
waypoint_follower:
  ros__parameters:
    waypoint_task_executor_plugin: "wait_at_waypoint"

# 특정 웨이포인트 실행 플러그인 설정
wait_at_waypoint:
  plugin: "nav2_waypoint_follower::WaitAtWaypoint"
  enabled: True
  waypoint_pause_duration: 200 # 대기 시간 설정
```

Behavior Server

구조



실행

주요 구성 요소

🔧 목적: Recovery & Action

BT Navigator의 요청에 따라 충돌 회피, 간헐 탈출 등의 복구 행동(Spin, Backup 등)을 수행합니다.

📦 패키지 및 플러그인

`nav2_behaviors` 패키지의 `BehaviorServer`를 Lifecycle 노드로 실행합니다.

⚙️ 파라미터 및 TF

플러그인 목록과 코스트맵 토폴 설정을 로딩하며, 네임스페이스 호환을 위해 `/tf`를 리매핑합니다.

```
<!-- Behavior Server: 복구 행동 실행 서버 -->
<composable_node pkg="nav2_behaviors"
  plugin="behavior_server::BehaviorServer"
  name="behavior_server">

  <!-- 파라미터 파일 로딩 -->
  <param from="$(var params_file)" />

  <!-- 공통 설정 -->
  <param name="use_sim_time" value="$(var use_sim_time)" />
  <param name="autostart" value="$(var autostart)" />

  <!-- TF Remapping -->
  <remap from="/tf" to="tf" />
  <remap from="/tf_static" to="tf_static" />

</composable_node>
```

플러그인 구조 (behavior_plugins)

🔗 플러그인 기반 유연성

`behavior_plugins` 리스트에 이름을 등록하여 사용할 행동을 선택적으로 활성화할 수 있습니다.

기본 제공 플러그인 5종

- ✓ **Spin**
제자리 회전 (장애물 스캔 및 heading 보정)
- ✓ **BackUp**
후진 이동 (충돌 상황 탈출)
- ✓ **DriveOnHeading**
경로 계획 없이 heading 방향 직진 (재배치)
- ✓ **Wait**
일시 정지 (상황 관망 및 안정화)
- ✓ **AssistedTeleop**
충돌 방지가 적용된 원격 제어 보조

```
behavior_server:
  ros_parameters:
    # 활성화할 플러그인 이름 목록 등록
    behavior_plugins: ["spin", "backup", "drive_on_heading", "assisted_teleop", "wait"]

    # --- 각 플러그인 타입 매핑 ---
  spin:
    plugin: "nav2_behaviors::Spin"

  backup:
    plugin: "nav2_behaviors::BackUp"

  drive_on_heading:
    plugin: "nav2_behaviors::DriveOnHeading"

  wait:
    plugin: "nav2_behaviors::Wait"

  assisted_teleop:
    plugin: "nav2_behaviors::AssistedTeleop"

  # 공통 파라미터 (좌표계 등)
  local_frame: odom
  global_frame: map
  robot_base_frame: base_footprint
```

Behavior Server의 공간 인지 기준이 되는 프레임 및 TF 허용 오차 정의

주요 파라미터

local_frame & global_frame

odom: 근거리 회피 및 연속성 보장을 위한 로컬 프레임
map: 전역 위치 추정을 위한 글로벌 프레임

robot_base_frame

로봇의 중심(풋프린트 기준점) 링크. 충돌 계산의 기준이 됨
(base_footprint)

transform_tolerance

TF(좌표 변환) 데이터 수신 지연에 대한 허용 오차 시간(초). 네트워크 지연 등을 고려해 0.1 초 설정

enable_stamped_cmd_vel

속도 명령 메시지 타입 선택 (false: Twist, true: TwistStamped)

```
nav2_params.yaml

behavior_server:
  ros__parameters:
    # --- Frame Settings ---
    local_frame: "odom"
    global_frame: "map"
    robot_base_frame: "base_footprint"

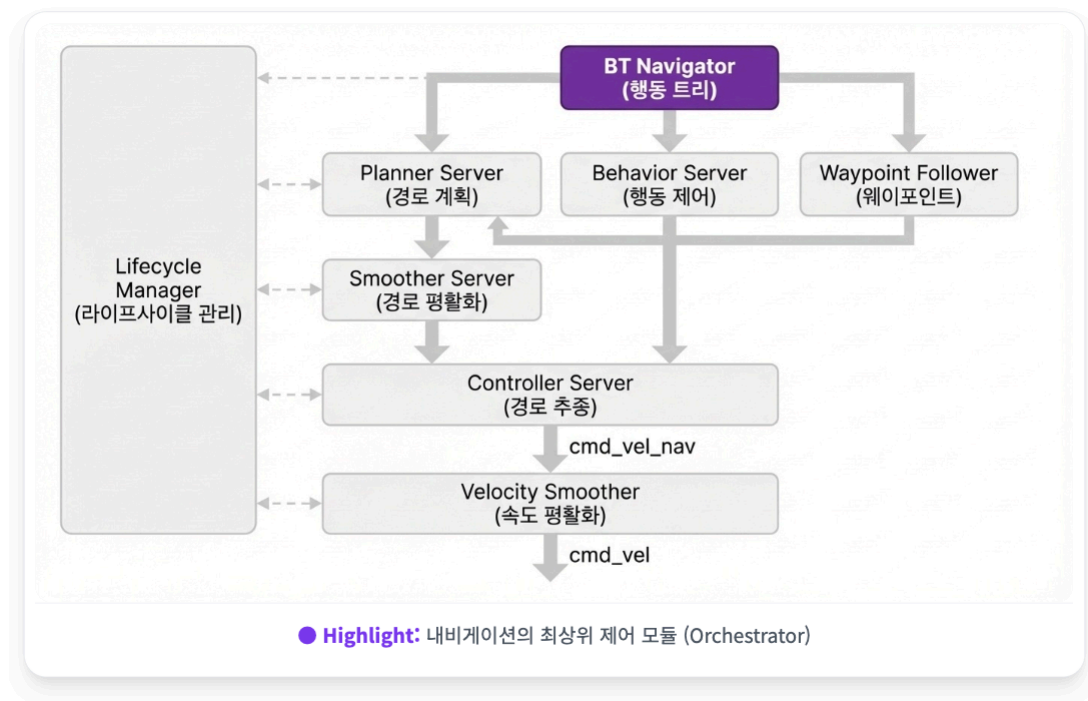
    # --- Transform Tolerance ---
    # TF lookup delay tolerance (seconds)
    transform_tolerance: 0.1

    # --- Command Output ---
    # Use TwistStamped instead of geometry_msgs/Twist
    enable_stamped_cmd_vel: false

    # --- Additional Costmap Topics ---
    local_costmap_topic: "local_costmap/costmap_raw"
    global_costmap_topic: "global_costmap/costmap_raw"
```

BT Navigator

구조



Behavior Tree 기반의 자율주행 미션 오케스트레이션 및 서버 호출 구조

계층적 지휘 구조 (Hierarchical Structure)

👑 최상위 지휘자 (Top-Level Orchestrator)

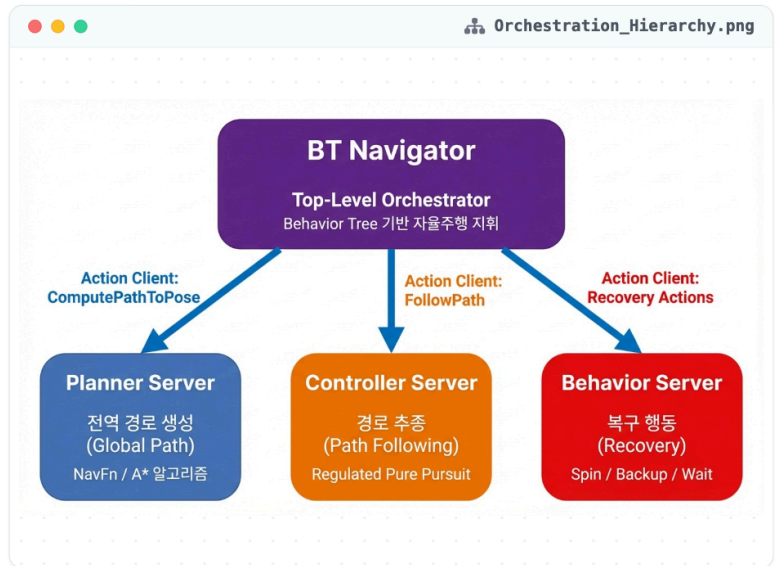
BT Navigator는 전체 미션의 논리적 흐름을 관장하며, 상황에 맞춰 하위 서버들에게 작업을 지시합니다. 직접 로봇을 제어하지 않고, '무엇을 해야 하는지'를 결정합니다.

🔧 ROS2 Action 기반 호출

각 서버와는 비동기적인 ROS2 Action Client 인터페이스를 통해 통신합니다. 이를 통해 작업의 성공/실패 여부를 모니터링하고 피드백을 수신합니다.

🖥️ 3대 핵심 서버 (Core Servers)

- **Planner**: 전역 경로 생성 (ComputePathToPose)
- **Controller**: 경로 추종 제어 (FollowPath)
- **Behavior**: 복구 행동 수행 (Recovery Actions)



내비게이션 스택의 최상위 오케스트레이터(Orchestrator) 실행 구성

🔧 핵심 역할

BT Navigator는 Behavior Tree(BT)를 로드하고 실행하여 전체 자율주행 미션을 관리합니다. Planner, Controller, Recovery 서버를 적절한 시점에 호출하고 상태를 모니터링합니다.

Launch 주요 포인트

✅ Node & Plugin

`nav2_bt_navigator` 패키지의 `BtNavigator` 클래스를 사용하여 실행됩니다.

⚠️ Cmd_vel Remapping

`cmd_vel` 을 `cmd_vel_nav` 로 리맵핑합니다. 이는 Velocity Smoother나 Multiplexer가 최종 속도 명령을 중재할 수 있도록 하기 위함입니다.

🔧 TF 호환성

네임스페이스를 사용하는 경우를 대비해 `/tf` 토픽을 상대 경로인 `tf` 로 리맵핑하여 유연성을 확보합니다.

```
navigation_launch.xml

<!-- BT Navigator 노드 실행 -->
<composable_node pkg="nav2_bt_navigator"
  plugin="nav2_bt_navigator::BtNavigator"
  name="bt_navigator">

  <!-- 파라미터 로딩 -->
  <param from="$(var params_file)"/>
  <param name="use_sim_time" value="$(var use_sim_time)"/>
  <param name="autostart" value="$(var autostart)"/>

  <!-- TF Topic Remapping -->
  <remap from="/tf" to="tf"/>
  <remap from="/tf_static" to="tf_static"/>

  <!-- Output Topic Remapping (중요!) -->
  <!-- Velocity Smoother로 출력 전달 -->
  <remap from="cmd_vel" to="cmd_vel_nav"/>

</composable_node>
```

BT Navigator의 좌표계 정의 및 실행 주기 설정

주요 파라미터 (Parameters)



좌표계 설정 (Frames)

`global_frame` 은 전역 경로 계획의 기준인 `map`을,
`robot_base_frame` 은 로봇의 기준점인 `base_link`를 지정합니다.



오도메트리 토픽 (Odometry)

로봇의 실시간 위치 및 속도 상태를 추정하기 위해 `odom` 토픽을 구독하여 현재 상태를 모니터링합니다.



BT 루프 주기 (Loop Duration)

Behavior Tree의 업데이트 간격(Tick)을 **10ms**로 설정하여 내비게이션 상태 변화에 빠르게 반응하도록 합니다.



ROS2 Navigation 실행해보기

가상 환경에서 로봇과 센서를 시뮬레이션하기 위해 Gazebo를 구동

TERMINAL 1 COMMAND

```
ros2 launch pinky_gz_sim launch_sim.launch.xml
```

📌 확인 포인트



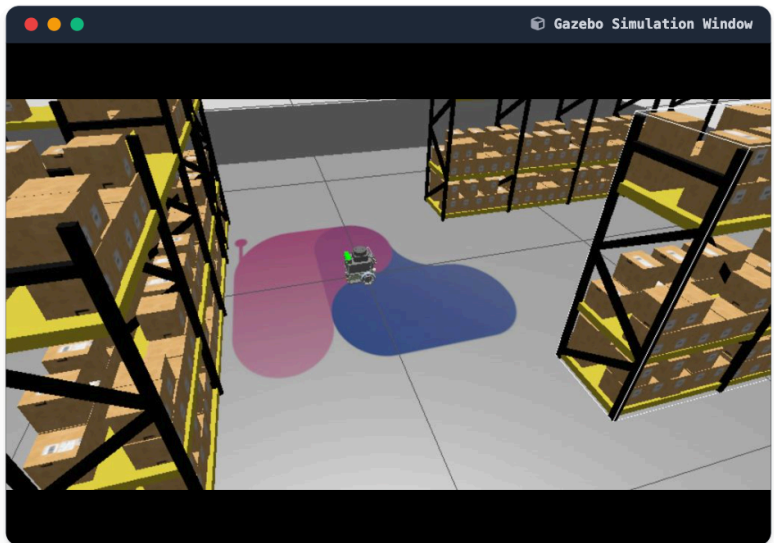
시뮬레이터 로딩

Gazebo 윈도우가 실행되고 창고(Warehouse) 월드와 로봇 모델이 정상적으로 렌더링되어야 합니다.



센서 데이터 발행

Lidar 스캐너가 회전하거나 레이저 포인트가 보여야 하며, 터미널에 센서 관련 오류가 없어야 합니다.



📹 실행 화면: 창고 환경에서의 로봇 배치 모습

터미널 2: Nav2 Bringup 실행 및 노드 생명주기(Lifecycle) 활성화 확인

1 작업 경로 이동

맵 파일이 위치한 디렉토리로 이동합니다.

```
cd ws/src/pinky_pro/pinky_navigation/map/
```

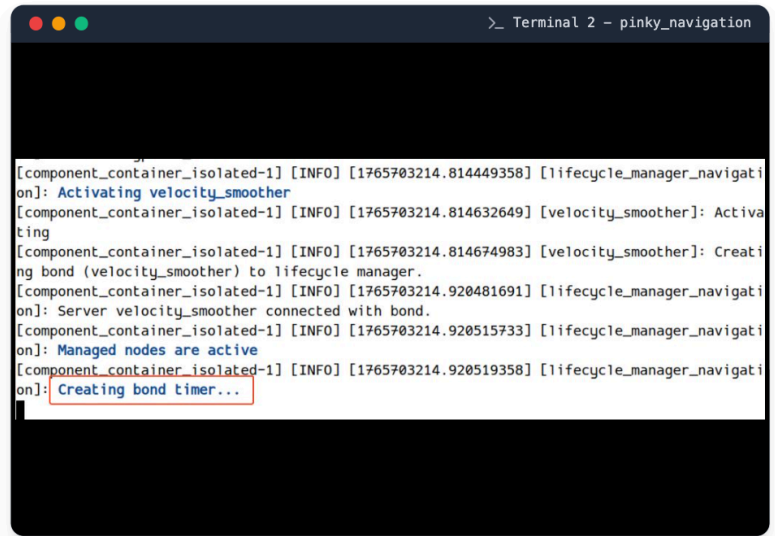
2 Navigation 실행

Bringup 런치 파일을 실행합니다. (맵, 시뮬레이션 시간 설정)

```
ros2 launch pinky_navigation bringup_launch.xml map:=my_map.yaml  
use_sim_time:=true
```

주요 확인 포인트

- ✓ 로그에 **"Creating bond timer..."** 문구가 출력되어야 함
- ✓ Lifecycle Manager가 관리 노드들을 **Active** 상태로 전환



※ Server velocity_smoother connected with bond 로그 확인 필수

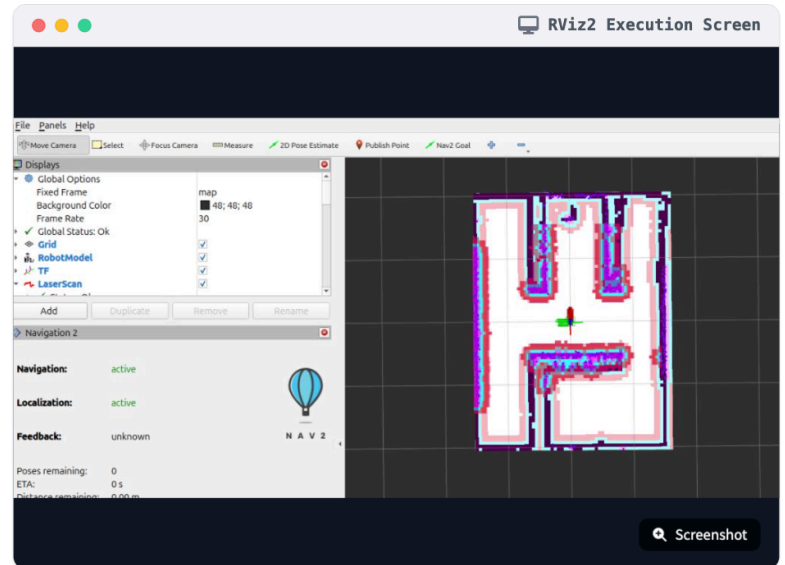
터미널3: 시각화 도구를 실행하여 로봇 상태를 모니터링하고 주행 명령 하달

실행 명령어 (Terminal 3)

```
ros2 launch pinky_navigation nav2_view.launch.xml
```

주요 확인 포인트

- ✓ **Nav2 패널 상태 확인**
좌측 패널에서 Navigation 및 Localization 상태가 **Active** (초록색)인지 확인
- ✓ **Map & TF 시각화**
지도가 정상 로딩되고, 로봇의 위치(TF)가 맵 상에 정확히 표시되는지 확인
- ✓ **Navigation Goal 명령**
'2D Goal Pose' 도구로 목표 지정 시 경로(Path)가 생성되고 주행이 시작되는지 확인



RViz2 인터페이스를 통한 목표 지점 지정과 자율주행 실행 과정

1 도구 선택 (Nav2 Goal)

RViz2 상단 툴바에 있는 **Nav2 Goal** 버튼을 클릭하여 목표 지점 설정 도구를 활성화합니다.

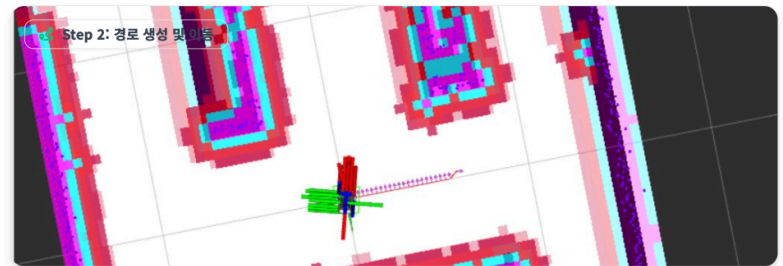
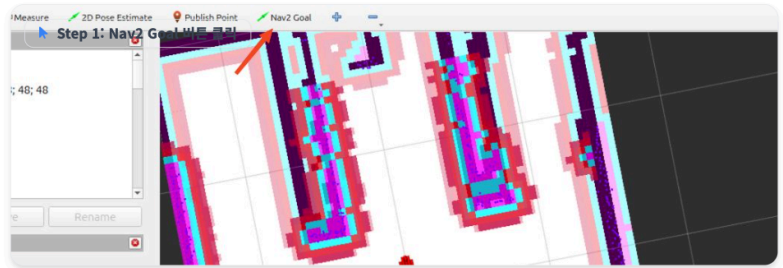
2 목표 지점 및 방향 지정

지도 상의 원하는 위치를 **클릭(Click)**한 상태로 마우스를 **드래그(Drag)**하여 로봇이 도착했을 때 바라볼 방향을 설정합니다.

화살표가 가리키는 방향이 로봇의 최종 헤딩(Heading)이 됩니다.

3 경로 생성 및 주행 확인

- ✓ Planner Server가 전역 경로(Path) 생성
- ✓ Controller Server가 속도 명령(cmd_vel) 출력
- ✓ 로봇이 경로를 따라 이동 시작



정말 수고하셨습니다.

MISSION COMPLETE

수고하셨습니다!

ROS2 Nav2의 방대한 과정을 성공적으로 마무리하신 것을 축하드립니다.

이제 여러분은 자율주행 로봇을 위한 강력한 내비게이션 시스템을
직접 구축하고 운용할 수 있는 전문가입니다.



핑크랩 스튜디오

@pinklab_studio

Official
Channel

Robotics & AI
Education

더 많은 로봇 공학 강의를 만나보세요



https://www.youtube.com/@pinklab_studio



PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio